

El Rastro

Segundo de Desarrollo de Aplicaciones Web

Nicolas Rodríguez Jiménez

15, 05, 2026



HOJA 0

Enlace a github:

<https://github.com/ElPirri00/EntregaFinal-El-Rastro>

Enlace a la web donde se ha desplegado la aplicación: local

Enlace a github del pdf de la memoria:

Enlace a github del pdf del manual de usuario de la aplicación:

<https://github.com/ElPirri00/Manual-Usuario-El-Rastro>

Usuarios creados para el uso de la aplicación:

Usuario normal:

Email: ana@demo.com

Contraseña: 123456

Usuario normal:

Email: pedro@demo.com

Contraseña: 123456

Usuario administrador:

Email: admin@demo.com

Contraseña: 123456

Enlace a github de los scripts para crear e inicializar la base de datos:

<https://github.com/ElPirri00/EntregaFinal-El-Rastro/tree/main/database>

Tabla de contenido

1. Descripción del proyecto	3
1.1. Características generales	4
1.2. Objetivos y alcance	5
2. Análisis del sector / mercado.....	6
2.1. Prospectiva del Título en el sector.....	6
2.2. Evolución y tendencia del sector	7
2.3. Normativa y documentación técnica específica	7
3. Plan de ejecución.....	8
3.1. Diagrama/cronograma de flujo de procesos (Diagrama de Gantt)	8
3.2. Proceso de desarrollo software	9
3.2.1. Fase de análisis	10
3.2.1.1. Tipos de usuario.....	10
3.2.1.2. Descripción de requisitos	11
3.2.1.3. Casos de Uso.....	12
3.2.1.4. Guía de estilo	19
3.2.1.5. Prototipo del sitio web	21
3.2.1.6. Mapa de navegación.....	23
3.2.2. Fase de desarrollo.....	23
3.2.2.1. Base de datos.....	23
3.2.2.1.1. Análisis de requisitos de datos de la aplicación	23
a. Identificación de entidades e interrelaciones entre entidades	23
b. Identificación de atributos de cada entidad	23
3.2.2.1.2 Diseño lógico de datos.....	23
3.2.2.1.3 Paso del modelo lógico (E/R) al modelo relacional (tablas)	23
a. Identificación de atributos de cada tabla, sus tipos y tamaños.....	23
b. Identificación de claves principales y claves candidatas	23
c. Identificación de las reglas de integridad y seguridad de la base de datos	23
d. Modelo relacional (tablas).....	23

3.2.2.1.4 Aplicación de reglas de normalización al modelo relacional.....	23
3.2.2.1.5 Tipos de datos para el sistema gestor seleccionado	23
3.2.2.1.6 Scripts de creación de tablas e inserciones iniciales	23
3.2.2.2. Servidor.....	23
3.2.2.2.1. Lista de funciones en php	23
3.2.2.3. Cliente.....	23
3.2.2.3.1. Diseño de la interfaz	23
3.2.2.3.2. Accesibilidad	23
3.2.2.3.3. Usabilidad	23
3.2.2.3.4. Desarrollo web entorno cliente.....	23
a. Formularios y su validación	23
b. Manejo y gestión de eventos: Teclado, ratón y estados de la ventana ..	23
c. Gestión y almacenamiento de datos e información en el cliente	23
d. Modificación del DOM.....	23
e. Animaciones, efectos, cambios dinámicos de estilos etc... para dinamizar la parte visible al cliente	23
f. Comunicación AJAX.....	23
g. Comunicación asíncrona con el servidor	23
3.2.3. Fase de despliegue.....	23
3.2.3.1. Despliegue utilizando un hosting	23
3.2.3.2. Despliegue en un equipo local propio	23
3.3. Seguimiento y control de incidencias	23
3.4. Indicadores de calidad de procesos.....	23
4. Recursos materiales.....	23
4.1. Inventario, valorado de medios	23
4.2. Presupuesto económico	23
5. Recursos humanos.....	23
5.1. Organización	23
5.2. Contratación	23
5.3. Prevención de riesgos laborales	23

6. Viabilidad técnica.....	23
6.1. Estudio de viabilidad técnica	23
7. Viabilidad económica-financiera	23
7.1. Inversiones y gastos	23
7.2. Financiación	23
7.3. Viabilidad económico-financiera	23
8. Conclusión.....	23
Bibliografía/Webgrafía.....	23
Anexos.....	23

1. Descripción del proyecto

El proyecto **El Rastro** consiste en el desarrollo de una aplicación web tipo marketplace destinada a la compraventa de productos de segunda mano entre usuarios.

La idea principal de la aplicación es ofrecer una plataforma sencilla donde los usuarios puedan registrarse, publicar productos, consultar anuncios de otros usuarios, contactar con vendedores, comprar productos y realizar valoraciones después de una compra.

Además, se ha organizado siguiendo una arquitectura **MVC**, separando los modelos, vistas y controladores para mantener el código más ordenado.

La aplicación se ejecuta en un entorno local mediante **XAMPP**, utilizando Apache como servidor web y MySQL como sistema gestor de base de datos.

1.1. Características generales

Las características principales de la aplicación son las siguientes:

- Registro e inicio de sesión de usuarios
- Publicación de productos de segunda mano
- Subida de imágenes de productos
- Listado y búsqueda de productos
- Página de detalle de producto con carrusel de imágenes
- Sistema de compra
- Sistema de mensajería entre usuarios
- Actualización automática de mensajes mediante JavaScript
- Valoraciones después de una compra
- Perfil de usuario
- Panel básico de administración
- Eliminación lógica de productos
- Bloqueo y desbloqueo de usuarios

La aplicación permite que un usuario registrado pueda publicar productos indicando datos como título, descripción, precio, categoría y estado del producto. También puede subir imágenes asociadas al anuncio.

Los compradores pueden consultar los productos disponibles, ver su detalle, contactar con el vendedor y realizar la compra. Después de comprar, pueden valorar al vendedor mediante una puntuación y un comentario.

También se incluye un panel de administración básico desde el que un administrador puede ocultar productos o bloquear usuarios.

1.2. Objetivos y alcance

El objetivo principal de **El Rastro** es ofrecer una plataforma web sencilla para facilitar la compraventa de productos de segunda mano entre usuarios.

La aplicación busca que cualquier persona pueda publicar artículos que ya no utiliza y que otros usuarios puedan encontrarlos, contactar con el vendedor y realizar una compra de forma sencilla.

Los objetivos principales de la web son:

- Facilitar la publicación de productos de segunda mano.
- Permitir a los usuarios buscar productos disponibles.
- Mostrar información clara de cada producto.
- Permitir la comunicación entre comprador y vendedor.
- Favorecer la reutilización de productos.
- Ofrecer un sistema básico de compra dentro de la plataforma.
- Permitir valorar a los usuarios después de una compra.
- Aumentar la confianza mediante opiniones y valoraciones.
- Permitir una gestión básica de productos y usuarios mediante un administrador.
- Crear una experiencia sencilla, clara y fácil de usar.

El alcance de la aplicación incluye las funcionalidades básicas necesarias para un marketplace: registro de usuarios, inicio de sesión, publicación de productos, subida de imágenes, listado de productos, detalle del producto, mensajería, compra simulada, valoraciones y administración básica.

Como posibles ampliaciones futuras se podrían añadir pagos reales, geolocalización, favoritos, filtros avanzados, notificaciones en tiempo real, publicidad y anuncios destacados de pago.

2. Análisis del sector / mercado

Las plataformas de compraventa de segunda mano han crecido mucho durante los últimos años debido al aumento del comercio electrónico y al interés por reutilizar productos.

Este tipo de aplicaciones permiten que los usuarios puedan vender artículos que ya no utilizan y que otros usuarios puedan comprarlos a un precio más bajo que un producto nuevo. Además, fomentan un consumo más sostenible, ya que alargan la vida útil de los productos.

Aplicaciones como Wallapop, Milanuncios, Vinted o eBay son ejemplos de plataformas que permiten comprar y vender productos entre particulares. El proyecto **El Rastro** se inspira en este tipo de plataformas, pero con una versión más sencilla

2.1. Prospectiva del Título en el sector

El proyecto **El Rastro** se sitúa dentro del sector de las plataformas digitales de compraventa de productos de segunda mano. Este tipo de aplicaciones tienen una presencia cada vez mayor, ya que permiten a los usuarios vender productos que ya no utilizan y comprar artículos a precios más económicos.

La prospectiva del proyecto es positiva, ya que existe un interés creciente por el consumo responsable, la reutilización de productos y el ahorro económico. Cada vez más usuarios utilizan plataformas online para comprar y vender artículos entre particulares, especialmente en categorías como tecnología, moda, hogar, deporte y objetos de uso cotidiano.

El Rastro podría tener cabida dentro de este sector como una plataforma sencilla, local y fácil de utilizar. Su enfoque se basa en facilitar la publicación de anuncios, el contacto entre comprador y vendedor, la compra de productos y la valoración de usuarios para generar confianza.

Aunque existen plataformas consolidadas como Wallapop, Vinted o Milanuncios, el proyecto puede diferenciarse como una propuesta inicial más simple y cercana, orientada a usuarios que buscan una experiencia directa sin demasiada complejidad.

2.2. Evolución y tendencia del sector

El comercio electrónico ha evolucionado mucho en los últimos años. Cada vez más usuarios compran y venden productos a través de Internet, tanto en tiendas online como en plataformas entre particulares.

Una tendencia importante es el crecimiento del mercado de segunda mano. Muchos usuarios buscan productos más económicos y también existe una mayor conciencia sobre la sostenibilidad y la reutilización.

2.3. Normativa y documentación técnica específica

Para el desarrollo de la aplicación se han tenido en cuenta aspectos básicos relacionados con la seguridad, la protección de datos y las buenas prácticas en aplicaciones web.

En una aplicación real sería necesario tener en cuenta normativas como:

- Reglamento General de Protección de Datos (RGPD)
- Ley de Servicios de la Sociedad de la Información y Comercio Electrónico - (LSSI-CE)
- normativa sobre comercio electrónico
- política de privacidad
- aviso legal
- política de cookies

En este proyecto académico no se ha implementado una gestión legal completa, pero sí se han aplicado medidas técnicas básicas como:

- uso de contraseñas cifradas con `password_hash()`
- comprobación de contraseñas con `password_verify()`
- uso de sesiones PHP

- consultas preparadas con PDO
- validación de formularios en servidor
- control de acceso a zonas privadas
- control de permisos de administrador

3. Plan de ejecución

El desarrollo del proyecto se ha realizado de forma progresiva, dividiendo el trabajo en distintas fases.

Primero se definió la idea general de la aplicación y sus funcionalidades principales.

Después se diseñó la base de datos, la estructura MVC y la interfaz visual.

Posteriormente se desarrollaron las funcionalidades principales y se realizaron pruebas para corregir errores.

Las fases principales han sido:

- análisis del proyecto
- diseño de base de datos
- diseño de interfaz
- desarrollo backend/frontend
- pruebas
- documentación

3.1. Diagrama/cronograma de flujo de procesos (Diagrama de Gantt)

Fase	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6
Análisis de requisitos	X					
Diseño de base de datos	X	X				
Diseño de interfaz		X	X			
Desarrollo estructura MVC			X			

Desarrollo productos		X	X		
Desarrollo usuarios y sesiones		X	X		
Desarrollo compras y valoraciones			X	X	
Desarrollo mensajería			X	X	
Panel de administración				X	
Pruebas y correcciones				X	X
Documentación	X	X	X	X	X

3.2. Proceso de desarrollo software

El proceso de desarrollo seguido ha sido incremental. Primero se desarrollaron las funcionalidades principales y después se fueron añadiendo mejoras.

La aplicación se ha construido siguiendo una estructura MVC, que divide el proyecto en tres partes:

- Modelo: acceso y gestión de datos.
- Vista: interfaz que ve el usuario.
- Controlador: lógica que conecta modelos y vistas.

Esta separación permite que el código sea más ordenado y facilita el mantenimiento del proyecto.

El proceso seguido fue:

1. Definición de requisitos.
2. Diseño de entidades y base de datos.
3. Creación de la estructura de carpetas MVC.
4. Desarrollo del sistema de usuarios.
5. Desarrollo del sistema de productos.
6. Desarrollo de compras.
7. Desarrollo de mensajes.
8. Desarrollo de valoraciones.
9. Desarrollo del panel de administración.
10. Pruebas y corrección de errores.
11. Documentación final.

3.2.1. Fase de análisis

En la fase de análisis se definieron las funcionalidades necesarias para que la aplicación cumpliera su objetivo principal: permitir la compraventa de productos de segunda mano entre usuarios.

Se identificaron los tipos de usuario, los requisitos funcionales y los requisitos no funcionales.

También se decidió la estructura general de la base de datos y las entidades principales del sistema.

3.2.1.1. Tipos de usuario

En la aplicación existen principalmente dos tipos de usuario:

- Usuario registrado
- Administrador

Usuario registrado

Es el usuario normal de la aplicación. Puede realizar acciones como:

- registrarse
- iniciar sesión
- editar su perfil
- publicar productos
- editar sus productos
- eliminar sus productos
- ver productos publicados por otros usuarios
- contactar con vendedores
- comprar productos
- valorar vendedores después de una compra
- consultar sus mensajes

Administrador

Es un usuario con permisos adicionales. Puede acceder a un panel básico de administración.

Sus funciones principales son:

- ver todos los productos
- ocultar productos
- activar productos ocultos

- ver todos los usuarios
- bloquear usuarios
- desbloquear usuarios

El administrador no sustituye al usuario normal, sino que añade funciones de control y moderación de la plataforma.

3.2.1.2. Descripción de requisitos

Los requisitos del proyecto se dividen en requisitos funcionales y no funcionales.

Requisitos funcionales

- RF01 - El sistema debe permitir el registro de usuarios.
- RF02 - El sistema debe permitir iniciar y cerrar sesión.
- RF03 - El sistema debe permitir publicar productos.
- RF04 - El sistema debe permitir subir imágenes de productos.
- RF05 - El sistema debe mostrar un listado de productos disponibles.
- RF06 - El sistema debe permitir buscar productos.
- RF07 - El sistema debe mostrar el detalle de cada producto.
- RF08 - El sistema debe mostrar un carrusel de imágenes en el detalle del producto.
- RF09 - El sistema debe permitir contactar con el vendedor de un producto.
- RF10 - El sistema debe permitir enviar y recibir mensajes.
- RF11 - El sistema debe actualizar los mensajes automáticamente cada pocos segundos.
- RF12 - El sistema debe permitir comprar un producto disponible.
- RF13 - El sistema debe marcar un producto como vendido tras la compra.
- RF14 - El sistema debe permitir valorar al vendedor después de una compra.
- RF15 - El sistema debe mostrar opiniones recientes del vendedor.
- RF16 - El sistema debe permitir editar el perfil del usuario.
- RF17 - El sistema debe permitir editar productos propios no vendidos.
- RF18 - El sistema debe permitir eliminar productos de forma lógica.
- RF19 - El sistema debe permitir acceder a un panel de administración.
- RF20 - El administrador debe poder ocultar o activar productos.
- RF21 - El administrador debe poder bloquear o desbloquear usuarios.

Requisitos no funcionales

- RNF01 - La aplicación debe estar desarrollada con PHP, MySQL, HTML5, CSS3 y JavaScript.

- RNF02 - La aplicación debe seguir una estructura MVC.
- RNF03 - La interfaz debe ser sencilla y responsive.
- RNF04 - La aplicación debe ejecutarse en entorno local con XAMPP.
- RNF05 - Las contraseñas deben almacenarse cifradas.
- RNF06 - Las consultas a base de datos deben realizarse con PDO.
- RNF07 - Las rutas deben centralizarse mediante un router.
- RNF08 - El código debe estar organizado por carpetas.
- RNF09 - El sistema debe mostrar mensajes de error o éxito al usuario.
- RNF10 - Los formularios deben validarse en servidor.

3.2.1.3. Casos de Uso

Identificador CU	CU_01
Nombre CU	Registro de usuario
Actores	Usuario no registrado
Descripción	Permite a un visitante registrarse en la plataforma creando una cuenta.
Prioridad	10
Precondición	El usuario no debe estar registrado en el sistema.
Datos de entrada	Nombre, email, contraseña, direccion, metodo de pago
Datos de salida	Usuario registrado en el sistema
Secuencia Normal	
Usuario	Sistema
1. Accede a la opcion de registro	
	2. Muestra formulario de datos
3. Introduce datos	
	4. Valida datos
5. Envía datos	
	6. Crea usuario y muestra información
Flujos alternativos	

Usuario	Sistema
	3.Introduce datos errores o repetidos a.Solicita correccion
Postcondición	Usuario registrado correctamente
CU Relacionados	CU_02

Identificador CU	CU_02
Nombre CU	Iniciar sesion
Actores	Usuario registrado
Descripción	Permite al usuario acceder a su cuenta
Prioridad	10
Precondición	Usuario registrado
Datos de entrada	Email y contraseña
Datos de salida	Sesion iniciada
Secuencia Normal	
Usuario	Sistema
1. Accede a login	2. Muestra formulario
3. Introduce credenciales	4.Valida datos
	5.Inicia sesion
Flujos alternativos	
Usuario	Sistema
3.Credenciales incorrectas	a.Muestra error
Postcondición	Usuario autenticado
CU Relacionados	CU_01

Identificador CU	CU_03
Nombre CU	Publicar producto
Actores	Usuario registrado
Descripción	Permite al usuario publicar un producto
Prioridad	9
Precondición	Usuario autenticado
Datos de entrada	Título, descripción, precio, imágenes
Datos de salida	Producto publicado
Secuencia Normal	
Usuario	Sistema
1.Accede a publicar producto	2.Muestra formulario
3.Introduce datos	4.Valida

5.Envía	6.Guarda producto
	7.Publica anuncio
Flujos alternativos	
Usuario	Sistema
3.Datos incompletos	3a.muestra error
Postcondición	Producto disponible para compra
CU Relacionados	Cu_05

Identificador CU	CU_04
Nombre CU	Buscar productos
Actores	Usuario
Descripción	Permite buscar productos
Prioridad	10
Precondición	ninguna
Datos de entrada	Texto de búsqueda, filtros
Datos de salida	Lista de productos
Secuencia Normal	
Usuario	Sistema
1.Introduce búsqueda	2.Procesa
	3.Muestra resultados
Flujos alternativos	
Usuario	Sistema
	3.No hay resultado
Postcondición	Productos mostrados
CU Relacionados	CU_05

Identificador CU	CU_05
Nombre CU	Ver detalles de producto
Actores	usuario
Descripción	Muestra la información detalla del producto
Prioridad	10
Precondición	Producto existente
Datos de entrada	ID producto
Datos de salida	Información del producto
Secuencia Normal	

Usuario	Sistema
1.Selecciona producto	2.Muestra detalles
Flujos alternativos	
Usuario	Sistema
	Producto eliminado
	Muestra error
Postcondición	Información mostrada
CU Relacionados	CU_04

Identificador CU	CU_06
Nombre CU	Comprar producto
Actores	Usuario
Descripción	Permite comprar un producto
Prioridad	10
Precondición	Usuario registrado, producto disponible
Datos de entrada	Método de pago
Datos de salida	Compra realizada
Secuencia Normal	
Usuario	Sistema
1.Selecciona comprar	2.Muestra opciones
3.Introduce datos	4.Procesa pago
	5.Confirma compra
Flujos alternativos	
Usuario	Sistema
Pago fallido	Muestra error
Postcondición	Producto vendido
CU Relacionados	Cu_05 , CU_07

Identificador CU	CU-07
Nombre CU	Editar perfil
Actores	Usuario registrado
Descripción	Permite modificar datos del usuario.
Prioridad	7
Precondición	Usuario autenticado
Datos de entrada	Nuevos datos

Datos de salida	Perfil actualizado
Secuencia Normal	
Usuario	Sistema
1. Accede a perfil	2. Muestra datos
3. Modifica	4. Valida
5. Guarda	6. Actualiza
Flujos alternativos	
Usuario	Sistema
	4.Datos invalidos
	a.Mostrar error
Postcondición	Perfil actualizado
CU Relacionados	CU_02

Identificador CU	CU_08
Nombre CU	Chat entre usuarios
Actores	Comprador, vendedor
Descripción	Permite comunicarse entre usuarios
Prioridad	8
Precondición	Usuario autenticado
Datos de entrada	Mensaje
Datos de salida	Mensaje enviado
Secuencia Normal	
Usuario	Sistema
Escribe mensaje	Envía
	Guarda y muestra
Flujos alternativos	
Usuario	Sistema
Postcondición	Mensaje registrado
CU Relacionados	CU_05

Identificador CU	CU_09
Nombre CU	Valorar usuario
Actores	Usuario
Descripción	Permite valorar tras una compra o venta
Prioridad	6

Precondición	Compra realizada
Datos de entrada	Puntuación
Datos de salida	Valoración registrada
Secuencia Normal	
Usuario	Sistema
Introduce valoración	Guarda
Flujos alternativos	
Usuario	Sistema
Postcondición	Valoración guardada
CU Relacionados	CU_06

Identificador CU	CU_10
Nombre CU	Eliminar producto
Actores	Usuario
Descripción	Permite eliminar un producto publicado.
Prioridad	7
Precondición	Producto existente
Datos de entrada	ID producto
Datos de salida	Producto eliminado
Secuencia Normal	
Usuario	Sistema
Selecciona eliminar	Confirma
	Elimina
Flujos alternativos	
Usuario	Sistema
Producto vendido	No permite
Postcondición	Producto eliminado
CU Relacionados	CU_03

Identificador CU	CU_11
Nombre CU	Cierre de sesión
Actores	Usuario autenticado

Descripción	Permite al usuario cerrar su sesión activa
Prioridad	9
Precondición	El usuario debe haber iniciado sesión previamente
Datos de entrada	Acción de cerrar sesión
Datos de salida	Sesión finalizada
Secuencia Normal	
Usuario	Sistema
Selecciona la opción "Cerrar sesión"	Recibe solicitud
	Elimina datos de sesion
	Redirige al login
	Muestra mensaje de cierre correcto
Flujos alternativos	
Usuario	Sistema
Sesion ya cerrada	Redirige al login
Postcondición	Usuario desautenticado y sin sesión activa
CU Relacionados	CU_02

Identificador CU	CU-12
Nombre CU	Editar producto
Actores	Vendedor
Descripción	Permite modificar un producto publicado.
Prioridad	8
Precondición	Producto existente
Datos de entrada	Producto
Datos de salida	Producto actualizado
Secuencia Normal	
Usuario	Sistema
1. Accede a editar	2. Muestra datos
3.Modifica	4.Valida
5.Guarda	6.Actualiza
Flujos alternativos	
Usuario	Sistema
Datos inválidos	Error
Postcondición	Producto actualizado
CU Relacionados	Cu_03

3.2.1.4. Guía de estilo

1. Identidad visual

- Nombre del producto: El Rastro
- Concepto: Mercado de segunda mano digital, cercano, local y sostenible.
- Valores visuales:
 - Cercanía (colores suaves y cálidos)
 - Sostenibilidad (uso de verde)
 - Simplicidad (interfaces limpias)
 - Confianza (espacios amplios, bordes redondeados)

2. Paleta de colores

- Colores principales
 - Verde principal (CTA / branding):
Hex: #16A34A (aprox.)
Uso: botones principales, enlaces activos, elementos destacados
- Verde degradado (hero):
 - Inicio: #0F3D2E
 - Fin: #16A34A
 - Uso: banners principales

Colores neutros

- Fondos principales:
 - Gris claro:
#F5F5F5
- Fondos secundarios:
 - Gris medio:
#D1D5DB
- Bordes, inputs:
 - Gris oscuro (texto secundario):
#6B7280
 - Negro suave (texto principal):
#111827

Colores de estado

- Éxito: Verde principal
- Error: #DC2626
- Advertencia: #F59E0B

3. Tipografía

-Fuente principal

Sans-serif moderna (ejemplo: Inter / Poppins / Roboto)

Jerarquía:

-H1 (títulos grandes):

-28–32px

-SemiBold / Bold:

-H2 (secciones):

-22–24px

-SemiBold

Texto normal:

-14–16px

-Regular

Labels / inputs:

-12–14px

4. Layout y espaciado

Sistema de espaciado (base 8px)

4px (micro)

8px (base)

16px (espaciado estándar)

24px (secciones)

32px+ (bloques grandes)

Contenedores

Bordes redondeados: 12px – 16px

Sombras suaves:

box-shadow: 0 4px 12px rgba(0,0,0,0.08)

5. Componentes

Botón principal

-Fondo: verde

-Texto: blanco

-Bordes: redondeados (20px aprox.)

-Padding: 10px 20px

-Estados:

-Hover: verde más oscuro

-Disabled: gris claro

Botón secundario

-Fondo: blanco

- Borde: verde

- Texto: verde

Input / formularios

- Fondo: blanco

- Borde: gris claro

- Bordes redondeados: 8–10px

- Padding: 10px

- Focus:

- Borde verde

Cards (productos)

- Fondo: blanco

- Borde: gris claro

- Radio: 12px

- Contenido centrado

- Incluyen:

- Imagen

- Nombre

- Precio

6. Navegación

Navbar

- Elementos:

- Logo (izquierda)

- Buscador (centro)

- Botones (derecha)

- Botones:

- “Inicio” (activo en verde)

- “Vender”

- “Iniciar sesión”

7. Uso de imágenes

- Estilo ilustrativo

- Estilo neutro en productos

- Evitar sobrecarga visual

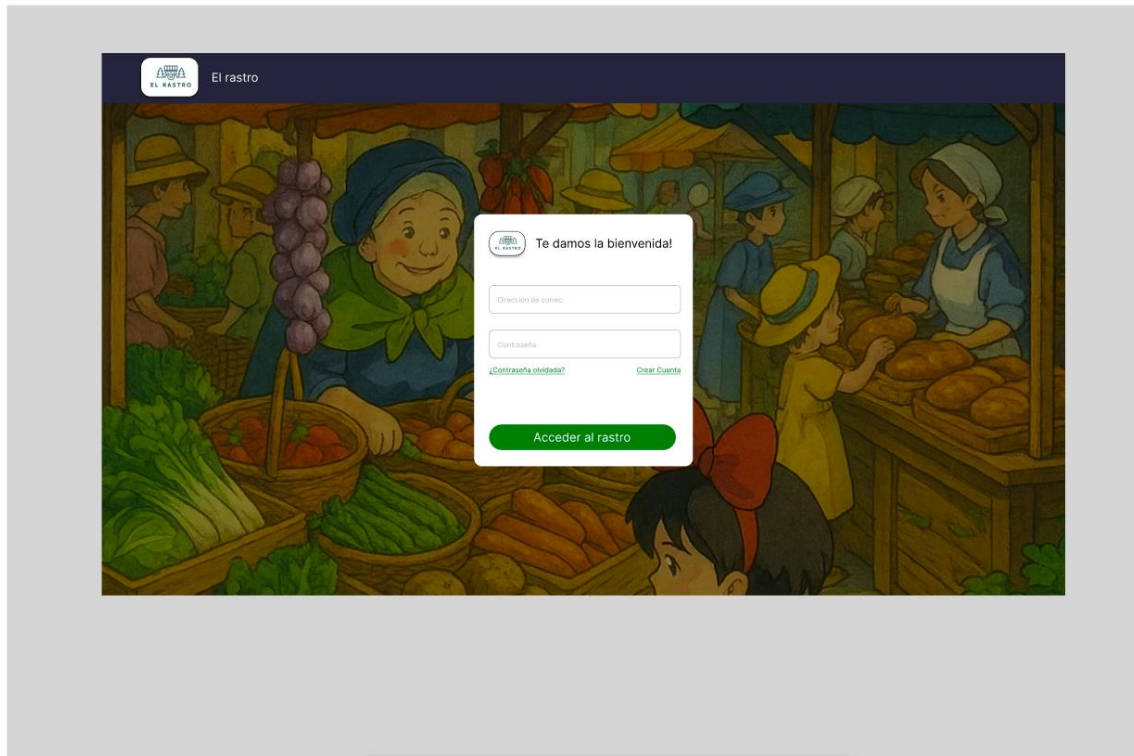
- Priorizar claridad

3.2.1.5. Prototipo del sitio web

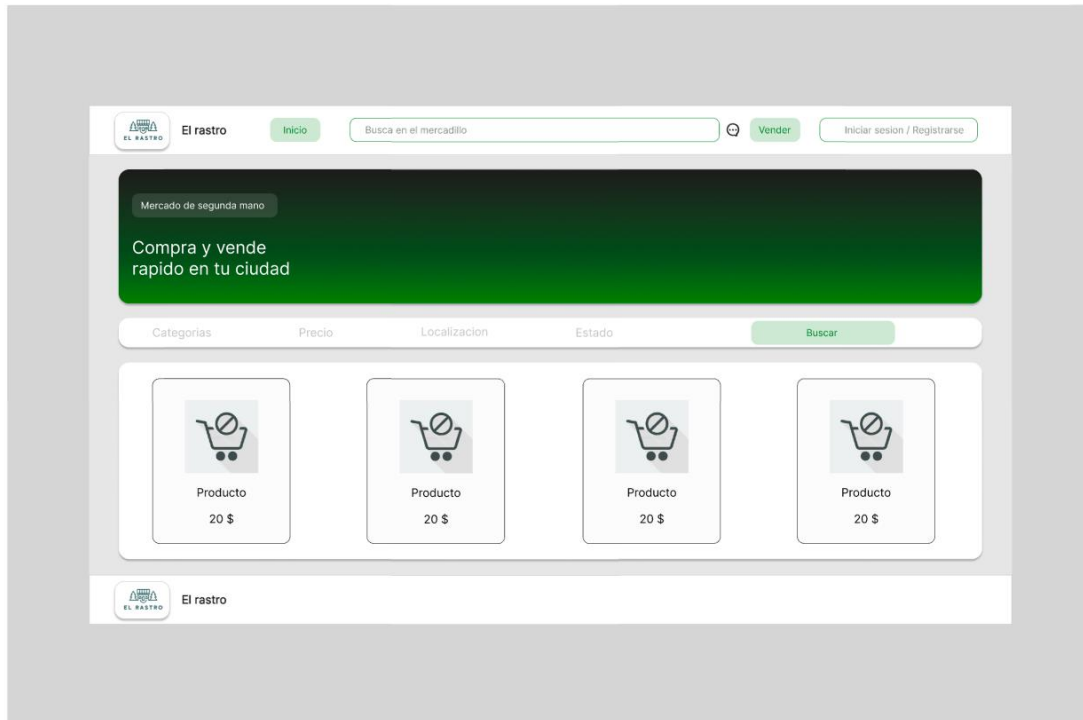
Link:

<https://www.figma.com/proto/ckkZtQrHiqsy7eSWoMwdMN/Untitled?node-id=0-1&t=G2eUlHZvWljCsgnO-1>

Login

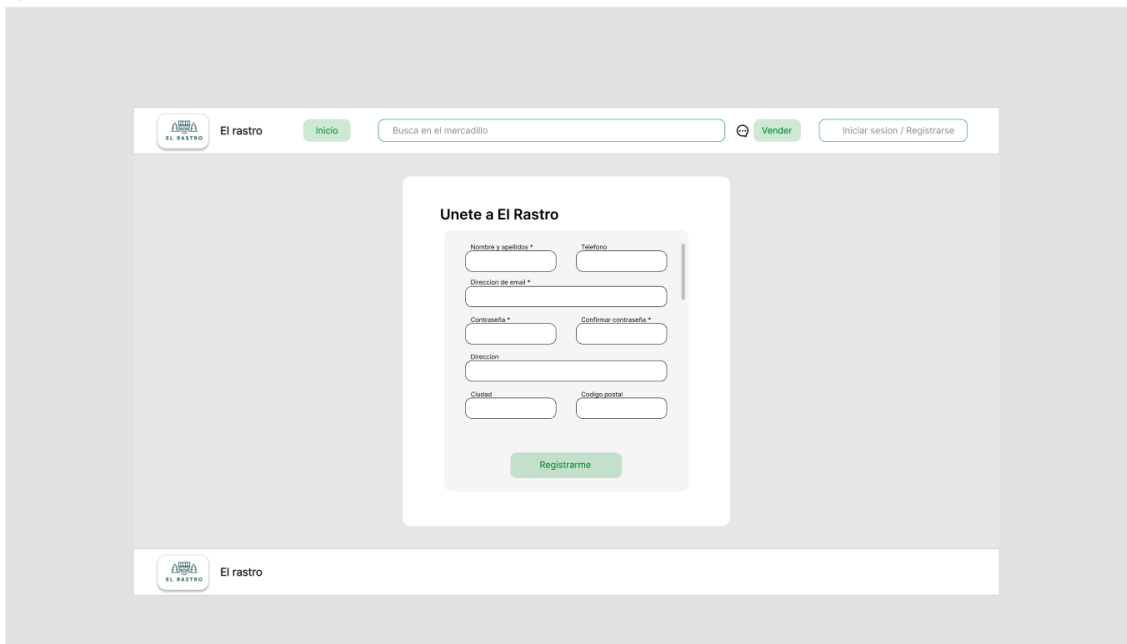


Index



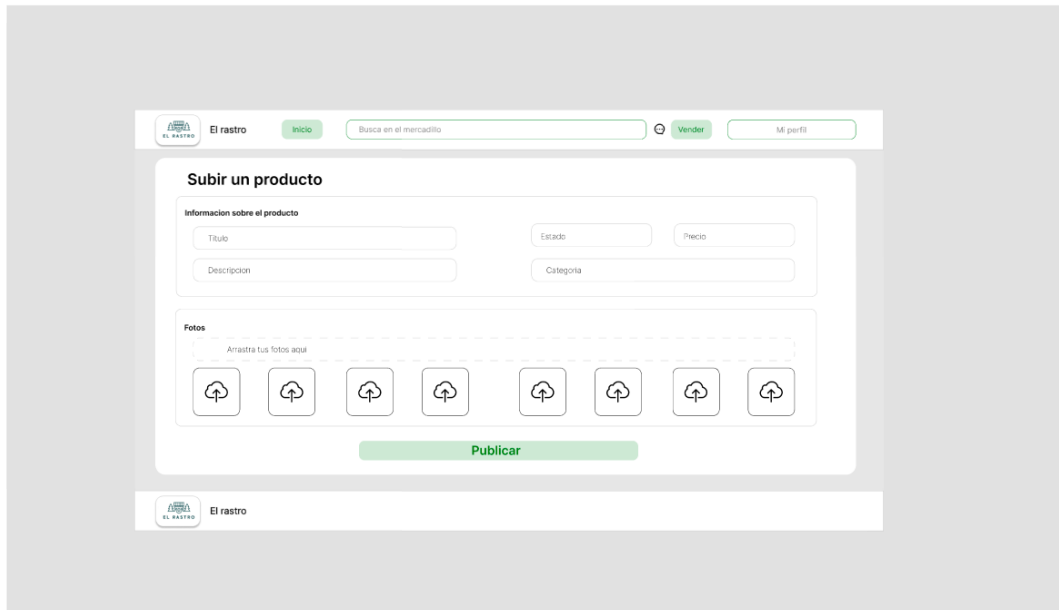
The Index page features a header with the 'El rastro' logo, a green 'Inicio' button, a search bar with the placeholder 'Busca en el mercadillo', a green 'Vender' button, and a link to 'Iniciar sesion / Registrarse'. Below the header is a green banner with the text 'Mercado de segunda mano' and 'Compra y vende rapido en tu ciudad'. Underneath the banner is a filter bar with tabs for 'Categorias', 'Precio', 'Localizacion', and 'Estado', followed by a green 'Buscar' button. The main content area displays four product cards, each with a shopping cart icon, the word 'Producto', and a price of '20 \$'. The footer contains the 'El rastro' logo and name.

Registro



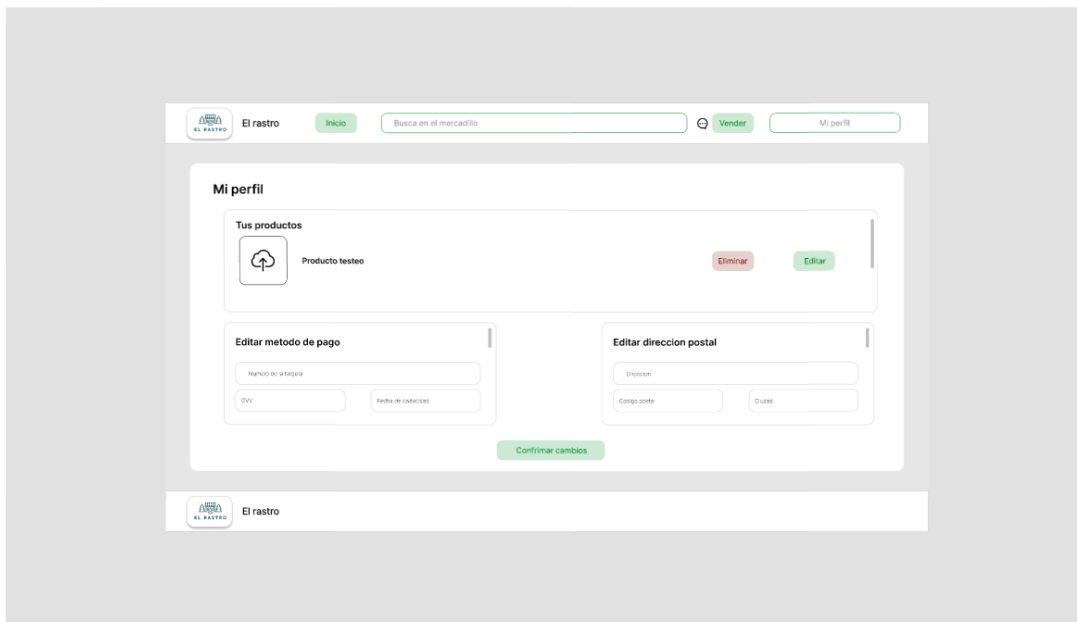
The Registro page features the same header as the Index page. The main content area is a registration form titled 'Unete a El Rastro'. The form includes input fields for 'Nombre y apellidos *', 'Telefono', 'Direccion de email *', 'Contraseña *', 'Confirmar contraseña *', 'Direccion', 'Ciudad', and 'Codigo postal'. A green 'Registrate' button is located at the bottom of the form. The footer contains the 'El rastro' logo and name.

Publicar/Vender



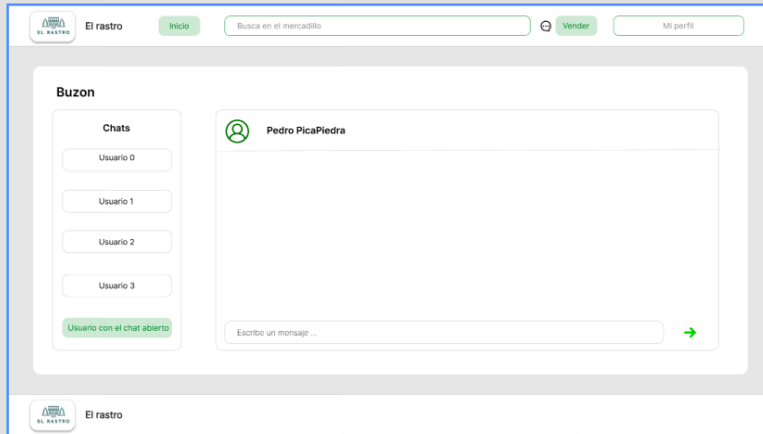
The screenshot shows the 'Subir un producto' form. At the top, there is a navigation bar with the 'El rastro' logo, a 'Inicio' button, a search bar with the placeholder 'Busca en el mercadillo', and buttons for 'Vender' and 'Mi perfil'. The form itself is titled 'Subir un producto' and contains several input fields: 'Titulo', 'Estado', 'Precio', 'Descripcion', and 'Categoria'. Below these fields is a section for 'Fotos' with a dashed box and the text 'Añade tus fotos aquí'. There are seven upload icons (clouds with arrows) in this section. At the bottom of the form is a green 'Publicar' button. The footer of the page shows the 'El rastro' logo and the text 'El rastro'.

Gestion de perfil



The screenshot shows the 'Mi perfil' page. The navigation bar is identical to the previous page. The main content area is titled 'Mi perfil' and contains three sections: 'Tus productos' (Your products), 'Editar metodo de pago' (Edit payment method), and 'Editar direccion postal' (Edit postal address). The 'Tus productos' section shows a single product card for 'Producto testeo' with an upload icon, and buttons for 'Eliminar' (Delete) and 'Editar' (Edit). The 'Editar metodo de pago' section has input fields for 'Numero de tarjeta', 'CVV', and 'Fecha de caducidad'. The 'Editar direccion postal' section has input fields for 'Direccion', 'Codigo postal', and 'Ciudad'. At the bottom of the form is a green 'Confirmar cambios' button. The footer of the page shows the 'El rastro' logo and the text 'El rastro'.

Mensajería



Producto


El rastro

Inicio

Vender

Mi perfil

Ficha / Comparar producto



Producto ejemplo
500\$

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras tincidunt nisi et orci euismod, at finibus tortor semper. Phasellus rutrum ipsum ac orci imperdiet, quis facilisis odio volutpat. Fusce consectetur blandit orci, non lacinia mauris suscipit nec. Morbi porta luctus egestas. Sed tristique libero vitae eleifend euismod. Curabitur consequat nulla vitae sapien fringilla, eget lacinia mauris scelerisque. Quisque egestas dolor pretium luctus imperdiet. Cras quam nisi, pretium in mollis in, iaculis non tortor. Nulla facilisi. Aenean felis ligula, aliquam a viverra eget, dignissim sit amet nulla. Ut d

Informacion del vendedor

Nombre Alex
Valoracion 5/5
Ciudad Cuenca

Enviar mensaje

Comprar


El rastro

Popup Compra


El rastro

Inicio

Vender

Mi perfil

Ficha / Comparar producto



Producto ejemplo
500\$

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras tincidunt nisi et orci euismod, at finibus tortor semper. Phasellus rutrum ipsum ac orci imperdiet, quis facilisis odio volutpat. Fusce consectetur blandit orci, non lacinia mauris suscipit nec. Morbi porta luctus egestas. Sed tristique libero vitae eleifend euismod. Curabitur consequat nulla vitae sapien fringilla, eget lacinia mauris scelerisque. Quisque egestas dolor pretium luctus imperdiet. Cras quam nisi, pretium in mollis in, iaculis non tortor. Nulla facilisi. Aenean felis ligula, aliquam a viverra eget, dignissim sit amet nulla. Ut d

Confirmar compra

Entrega
Se mostrará la información almacenada en el perfil de su dirección

Editar

Método de pago
Se mostrará la información almacenada en el perfil de su método de pago

Editar

Confirmar

Informacion del vendedor

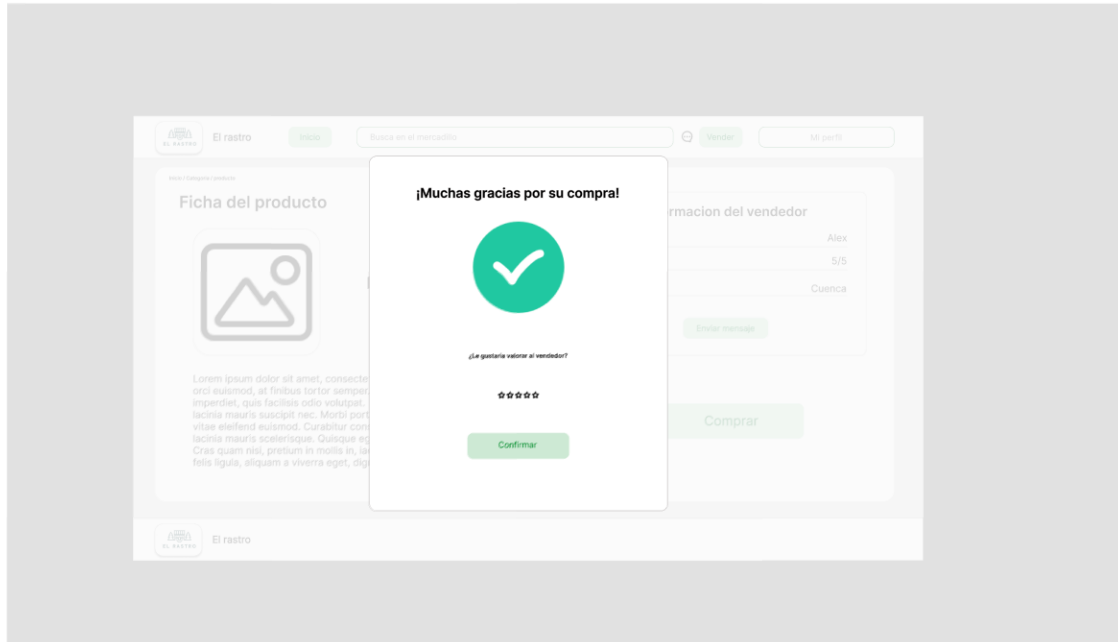
Nombre Alex
Valoracion 5/5
Ciudad Cuenca

Enviar mensaje

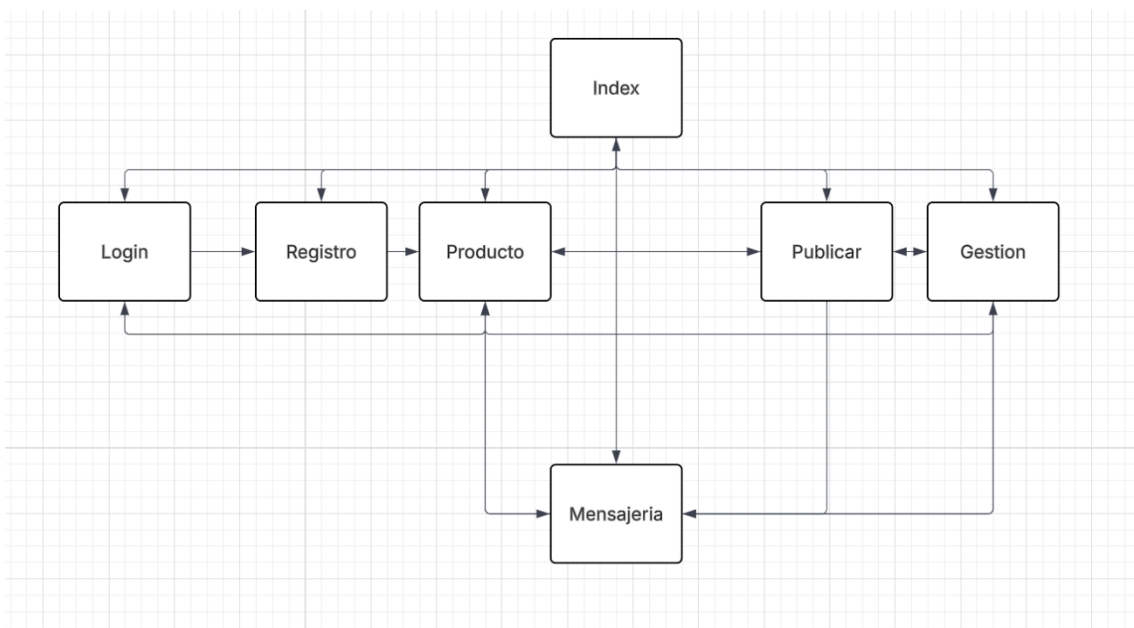
Comprar


El rastro

Popup Valoracion



3.2.1.6. Mapa de navegación



3.2.2. Fase de desarrollo

3.2.2.1. Base de datos

3.2.2.1.1. Análisis de requisitos de datos de la aplicación

a. Identificación de entidades e interrelaciones entre entidades

USUARIO — PUBLICA — PRODUCTO

Un usuario puede publicar muchos productos

Un producto pertenece a un solo usuario

1 : N

PRODUCTO — TIENE — IMAGEN

Un producto puede tener varias imágenes

Una imagen pertenece a un solo producto

1 : N

USUARIO — COMPRA — PRODUCTO (a través de COMPRA)

Un usuario puede comprar muchos productos

Un producto solo puede tener una compra (cuando se vende)

Relación resuelta con entidad COMPRA

USUARIO (1) — (N) COMPRA (N) — (1) PRODUCTO

USUARIO — ENVÍA — MENSAJE — RECIBE — USUARIO

Un usuario puede enviar muchos mensajes

Un usuario puede recibir muchos mensajes

Relación recursiva (usuario con usuario)

0,N:0,N

USUARIO — VALORA — USUARIO (a través de COMPRA)

Un usuario puede valorar a muchos usuarios

Un usuario puede recibir muchas valoraciones

Relación recursiva terciaria, ya que se valora al usuario pero a través de una compra

1:1

b. Identificación de atributos de cada entidad

USUARIO

Representa a cualquier persona registrada.

Atributos:

- id_usuario (PK)
- nombre
- email (único)
- contraseña
- dirección
- método_pago
- fecha_registro
- Tipo
- Activo

PRODUCTO

Artículo que se vende.

Atributos:

- id_producto (PK)
- título
- descripción
- precio
- categoría
- estado_producto
- estado (disponible, vendido, eliminado)
- fecha_publicación
- id_usuario (FK → USUARIO) (vendedor)

IMAGEN

Imágenes asociadas a un producto.

Atributos:

- id_imagen (PK)
- url
- id_producto (FK)

COMPRA

Registro de una transacción.

Atributos:

- id_compra (PK)
- fecha
- estado (pagado, fallido)
- metodo_pago
- id_comprador (FK → USUARIO)
- id_producto (FK → PRODUCTO)

MENSAJE

Chat entre usuarios.

Atributos:

- id_mensaje (PK)
- contenido
- fecha
- id_emisor (FK → USUARIO)
- id_receptor (FK → USUARIO)
- id_producto (FK → PRODUCTO)

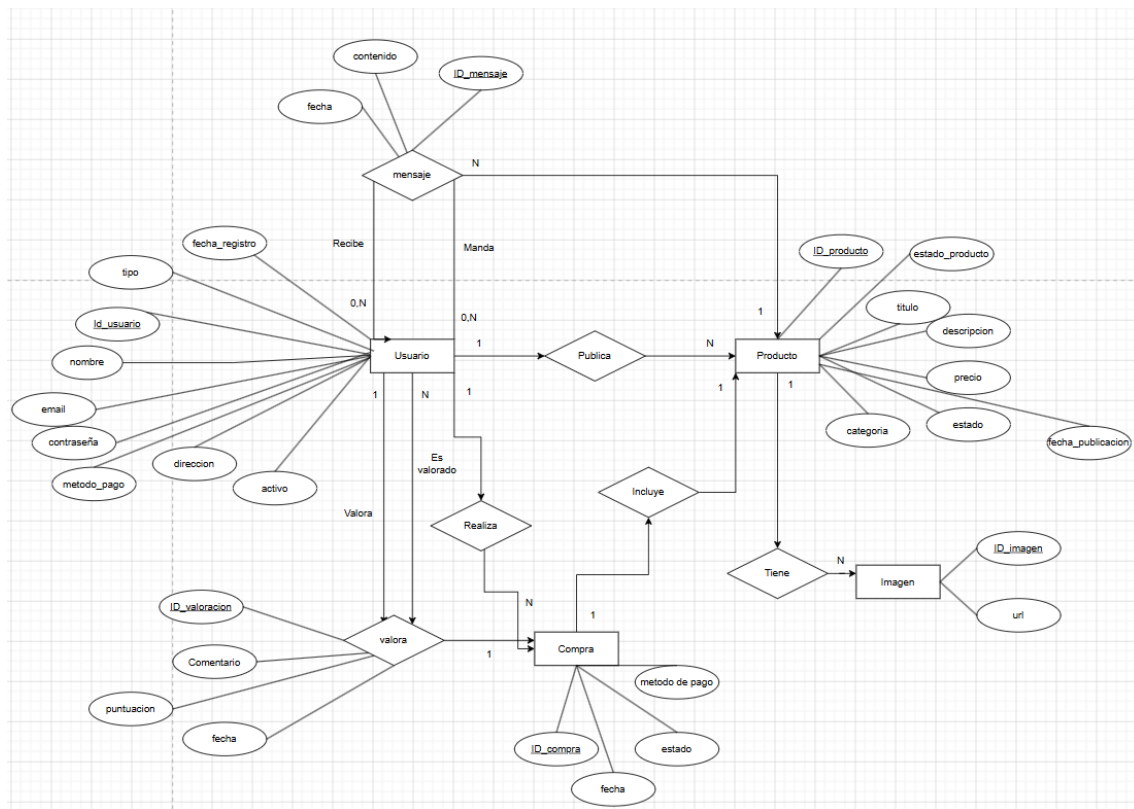
VALORACION

Valoraciones entre usuarios tras compra.

Atributos:

- id_valoracion (PK)
- puntuacion
- comentario (opcional)
- fecha
- id_emisor (FK → USUARIO)
- id_receptor (FK → USUARIO)
- id_compra (FK → COMPRA)

3.2.2.1.2 Diseño lógico de datos



3.2.2.1.3 Paso del modelo lógico (E/R) al modelo relacional (tablas)

a. Identificación de atributos de cada tabla, sus tipos y tamaños

Tabla Usuario

- id_usuario: Identificador único del usuario (clave primaria).
- nombre: Nombre del usuario.
- email: Correo electrónico único.
- contrasena: Contraseña cifrada.
- direccion: Dirección del usuario.
- metodo_pago: Método de pago asociado.
- fecha_registro: Fecha de creación de la cuenta.
- tipo: tipo de usuario.
- activo: indica si el usuario está activo o bloqueado.

Tabla Producto

- id_producto: Identificador único (PK).
- titulo: Nombre del producto.
- descripcion: Información detallada.

- precio: Precio del producto.
- categoria: Categoría del producto.
- estado_producto: Estado físico del producto.
- estado: Disponible, vendido o eliminado.
- fecha_publicacion: Fecha de publicación.
- id_usuario: Usuario vendedor (FK).

Tabla Imagen

- id_imagen: Identificador único (PK).
- url: Ruta de la imagen.
- id_producto: Producto asociado (FK).

Tabla Compra

- id_compra: Identificador único (PK).
- fecha: Fecha de compra.
- estado: Estado del pago.
- metodo_pago: Método utilizado.
- id_comprador: Usuario comprador (FK).
- id_producto: Producto comprado (FK).

Tabla Mensaje

- id_mensaje: Identificador único (PK).
- contenido: Texto del mensaje.
- fecha: Fecha de envío.
- id_emisor: Usuario que envía (FK).
- id_receptor: Usuario que recibe (FK).
- id_producto: Producto relacionado con el mensaje o conversación (FK).

Tabla Valoracion

- id_valoracion: Identificador único (PK).
- puntuacion: Valor numérico.
- comentario: Opinión del usuario.
- fecha: Fecha de valoración.
- id_emisor: Usuario que valora (FK).
- id_receptor: Usuario valorado (FK).
- id_compra: Compra asociada (FK).

b. Identificación de claves principales y claves candidatas

c. Identificación de las reglas de integridad y seguridad de la base de datos

Integridad referencial

Producto.id_usuario → Usuario.id_usuario
Imagen.id_producto → Producto.id_producto
Compra.id_comprador → Usuario.id_usuario
Compra.id_producto → Producto.id_producto
Mensaje.id_emisor / id_receptor → Usuario.id_usuario
Mensaje.id_producto → Producto.id_producto
Valoracion.id_emisor / id_receptor → Usuario.id_usuario
Valoracion.id_compra → Compra.id_compra

Integridad de unicidad

Producto.id_producto ->PK
Usuario.email ->único
Compra.id_producto ->único (un producto solo se vende una vez)
Valoracion.id_compra ->único (una compra solo se puede valorar una vez)
Todas las claves primarias -> únicas por definición

Integridad de semantica

precio -> número decimal positivo
Puntuacion ->valores entre 1 y 5
categoria -> {Electrónica, Moda, Hogar, Deportes, Otros}
estado_producto -> {Nuevo, Como nuevo, Usado, Reacondicionado}
estado (producto) -> {disponible, vendido, eliminado}
estado (compra) -> {pagado, fallido}
Tipo -> { administrador, usuario}
activo -> {0, 1}
email -> formato válido
fecha -> tipo fecha válida

Integridad de dominio

Un usuario no puede comprar su propio producto
Un usuario no puede automensajearse
Solo se puede valorar una compra si está completada
Un producto solo puede estar en estado “vendido” si existe una compra
No se puede valorar más de una vez la misma compra
Los mensajes deben tener sentido entre usuarios distintos

d. Modelo relacional (tablas)

USUARIO: (id_usuario PK, nombre, email UNIQUE, contrasena, direccion, metodo_pago, fecha_registro, tipo, activo)

PRODUCTO:(id_producto PK, titulo, descripcion, precio, categoria, estado_producto, estado, fecha_publicacion, id_usuario FK → USUARIO(id_usuario))

IMAGEN(id_imagen PK, url, id_producto FK → PRODUCTO(id_producto))

COMPRA:(id_compra PK, fecha, estado, metodo_pago, id_comprador FK → USUARIO(id_usuario), id_producto FK → PRODUCTO(id_producto) UNIQUE)

MENSAJE(id_mensaje PK, contenido, fecha, id_emisor FK → USUARIO(id_usuario), id_receptor FK → USUARIO(id_usuario), id_producto FK → PRODUCTO(id_producto))

VALORACION(id_valoracion PK, puntuacion, comentario, fecha, id_emisor FK → USUARIO(id_usuario), id_receptor FK → USUARIO(id_usuario), id_compra FK → COMPRA(id_compra))

3.2.2.1.4 Aplicación de reglas de normalización al modelo relacional

El modelo cumple:

- Primera Forma Normal (1FN)
 - No existen atributos multivalorados
 - Las imágenes se separan en su propia tabla
- Segunda Forma Normal (2FN)
 - Todas las tablas tienen clave primaria simple

- No existen dependencias parciales
- Tercera Forma Normal (3FN)
- No hay dependencias transitivas
- Cada atributo depende únicamente de su clave

3.2.2.1.5 Tipos de datos para el sistema gestor seleccionado

Tabla Usuario

Campo	Tipo de dato	Justificación
id_usuario	INT AUTO_INCREMENT	Identificador numérico único y autoincremental (PK)
nombre	VARCHAR(100)	Texto corto/medio
email	VARCHAR(150) UNIQUE	Correo único por usuario
contraseña	VARCHAR(255)	Contraseña cifrada (hash largo)
direccion	VARCHAR(255)	Dirección puede ser larga
metodo_pago	VARCHAR(50)	Tipo de pago asociado
fecha_registro	DATETIME	Fecha y hora de creación
tipo	VARCHAR(20)	Tipo de usuario
activo	TINYINT(1)	Indica si el usuario está activo o bloqueado

Tabla producto

Campo	Tipo de dato	Justificación
id_producto	INT AUTO_INCREMENT	PK
titulo	VARCHAR(150)	Nombre del producto
descripcion	TEXT	Texto largo sin límite fijo
precio	DECIMAL(10,2)	Valores monetarios exactos
categoria	VARCHAR(50)	Categoría del producto
estado_producto	VARCHAR(50)	Estado físico del producto
estado	VARCHAR(50)	Valores controlados
fecha_publicacion	DATETIME	Fecha completa
id_usuario	INT	FK a Usuario

Tabla Imagen

Campo	Tipo de dato	Justificación
id_imagen	INT AUTO_INCREMENT	PK
url	VARCHAR(255)	Ruta o enlace de la imagen
id_producto	INT	FK a Producto

Tabla compra

Campo	Tipo de dato	Justificación
id_compra	INT AUTO_INCREMENT	PK
fecha	DATETIME	Fecha de la compra
estado	VARCHAR(50)	Control de estado
metodo_pago	VARCHAR(50)	Forma de pago usada
id_comprador	INT	FK a Usuario
id_producto	INT	FK a Producto

Tabla Mensaje

Campo	Tipo de dato	Justificación
id_mensaje	INT AUTO_INCREMENT	PK
contenido	TEXT	Mensajes largos
fecha	DATETIME	Fecha de envío
id_emisor	INT	FK Usuario (emisor)
id_receptor	INT	FK Usuario (receptor)
id_producto	INT	FK Producto relacionado

Tabla Valoración

Campo	Tipo de dato	Justificación
id_valoracion	INT AUTO_INCREMENT	PK
puntuacion	INT	Valor pequeño
comentario	TEXT	Opinión libre
fecha	DATETIME	Fecha de valoración
id_emisor	INT	Usuario que valora
id_receptor	INT	Usuario valorado

id_compra INT FK a Compra

3.2.2.1.6 Scripts de creación de tablas e inserciones iniciales

Tabla Usuario:

```
CREATE TABLE Usuario (  
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  email VARCHAR(150) UNIQUE,  
  contraseña VARCHAR(255),  
  direccion VARCHAR(255),  
  metodo_pago VARCHAR(50),  
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  tipo VARCHAR(20) DEFAULT 'usuario',  
  activo TINYINT(1) NOT NULL DEFAULT 1  
);
```

Tabla producto:

```
CREATE TABLE Producto (  
  id_producto INT AUTO_INCREMENT PRIMARY KEY,  
  titulo VARCHAR(150),  
  descripcion TEXT,  
  precio DECIMAL(10,2),  
  categoria VARCHAR(50),  
  estado_producto VARCHAR(50),  
  estado VARCHAR(50),  
  fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  id_usuario INT,  
  FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario)  
);
```

Tabla Imagen

```
CREATE TABLE Imagen (  
  id_imagen INT AUTO_INCREMENT PRIMARY KEY,  
  url VARCHAR(255),  
  id_producto INT,  
  FOREIGN KEY (id_producto) REFERENCES Producto(id_producto)  
);
```

Tabla Compra

```
CREATE TABLE Compra (  
  id_compra INT AUTO_INCREMENT PRIMARY KEY,  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  estado VARCHAR(50),  
  metodo_pago VARCHAR(50),  
  id_comprador INT,  
  id_producto INT UNIQUE,  
  FOREIGN KEY (id_comprador) REFERENCES Usuario(id_usuario),  
  FOREIGN KEY (id_producto) REFERENCES Producto(id_producto)  
);
```

Tabla Mensaje

```
CREATE TABLE Mensaje (  
  id_mensaje INT AUTO_INCREMENT PRIMARY KEY,  
  contenido TEXT,  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  id_emisor INT,  
  id_receptor INT,  
  id_producto INT,  
  FOREIGN KEY (id_emisor) REFERENCES Usuario(id_usuario),  
  FOREIGN KEY (id_receptor) REFERENCES Usuario(id_usuario),  
  FOREIGN KEY (id_producto) REFERENCES Producto(id_producto)  
);
```

Tabla Valoracion

```
CREATE TABLE Valoracion (  
  id_valoracion INT AUTO_INCREMENT PRIMARY KEY,  
  puntuacion INT,
```



```
comentario TEXT,  
fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
id_emisor INT,  
id_receptor INT,  
id_compra INT,  
FOREIGN KEY (id_emisor) REFERENCES Usuario(id_usuario),  
FOREIGN KEY (id_receptor) REFERENCES Usuario(id_usuario),  
FOREIGN KEY (id_compra) REFERENCES Compra(id_compra)  
);
```

3.2.2.2. Servidor

La parte del servidor de la aplicación está desarrollada en **PHP** siguiendo una estructura **MVC**, separando la lógica de negocio, el acceso a datos y la presentación de la información.

El servidor se encarga de:

- recibir las peticiones del usuario;
- interpretar la ruta solicitada;
- consultar o modificar la base de datos;
- validar datos recibidos desde formularios;
- gestionar sesiones de usuario;
- controlar permisos;
- cargar las vistas correspondientes;
- devolver datos en formato JSON en el caso de la mensajería asíncrona.

La base de datos se gestiona mediante **PDO**, lo que permite realizar consultas preparadas y reducir el riesgo de inyección SQL.

3.2.2.2.1. Lista de funciones en php

Clase Database

La clase Database se encuentra en el núcleo de la aplicación y se encarga de crear y reutilizar la conexión con la base de datos.

`getConnection()`

Esta función se encarga de crear y devolver la conexión con la base de datos mediante PDO.

Se usa desde los modelos para poder hacer consultas a MySQL. Además, reutiliza la misma conexión si ya está creada, evitando abrir una conexión nueva cada vez.

Clase Router

`dispatch()`

Esta función lee la URL recibida por la aplicación y decide qué controlador y qué método se tienen que ejecutar.

Por ejemplo, una URL como:

`producto/detalle/5`

se interpreta como:

`ProductoController → detalle(5)`

Si no se indica ninguna ruta, carga por defecto el listado de productos.

Clase Controller

`view($view, $data = [])`

Carga una vista de la aplicación junto con el header y el footer.

También recibe un array de datos y los convierte en variables para poder usarlas dentro de la vista.

Por ejemplo, si se envía un producto desde el controlador, la vista puede mostrarlo directamente.

`redirect($route = "")`

Redirige al usuario a otra ruta de la aplicación.

Se utiliza después de acciones como iniciar sesión, cerrar sesión, publicar un producto o completar una compra.

`requireLogin()`

Comprueba si el usuario ha iniciado sesión.

Si no hay usuario guardado en sesión, redirige al login y muestra un mensaje de error.

`isPost()`

Comprueba si la petición actual se ha enviado mediante POST.

Se utiliza para saber cuándo un formulario ha sido enviado y debe procesarse.

Controladores

AuthController

login()

Gestiona el inicio de sesión.

Busca al usuario por email, comprueba la contraseña con `password_verify()` y, si todo es correcto, guarda los datos del usuario en la sesión.

También comprueba si la cuenta está activa. Si el usuario está bloqueado, no permite iniciar sesión.

registro()

Gestiona el registro de nuevos usuarios.

Valida que el nombre, email y contraseña sean correctos. Si todo está bien, crea el usuario en la base de datos.

La contraseña se guarda cifrada con `password_hash()`.

logout()

Cierra la sesión del usuario actual y lo redirige a la página de login.

ProductoController

__construct()

Carga los modelos de producto e imagen para poder usarlos en el resto del controlador.

index()

Muestra la página principal con el listado de productos disponibles.

También permite buscar productos mediante el parámetro q.

detalle(\$id)

Muestra la información completa de un producto.

Carga los datos del producto, sus imágenes y algunas valoraciones recientes del vendedor.

Si el producto no existe o está eliminado, redirige al listado principal.

vender()

Permite publicar un nuevo producto.

Comprueba que el usuario haya iniciado sesión, valida los datos del formulario y guarda el producto en la base de datos.

También guarda las imágenes subidas en la carpeta uploads y registra sus rutas en la tabla Imagen.

editar(\$id)

Permite editar un producto publicado por el usuario.

Comprueba que el producto exista y que pertenezca al usuario actual. Si los datos son válidos, actualiza el producto.

`eliminar($id)`

Elimina un producto de forma lógica.

No borra el registro de la base de datos, sino que cambia su estado a eliminado.

Esto evita problemas con compras, mensajes o valoraciones relacionadas.

`comprar($id)`

Gestiona la compra de un producto.

Comprueba que el producto esté disponible y que el usuario no sea el propio vendedor. Después valida el método de pago y crea la compra.

Cuando se realiza la compra, el producto pasa a estado vendido.

`validarPago($datos)`

Valida los datos introducidos en el formulario de pago.

Según el método elegido, comprueba campos distintos:

Tarjeta → titular, número, caducidad y CVV

PayPal → email

Bizum → teléfono

Efectivo → dirección o lugar para quedar

`guardarImagenes($idProducto)`

Guarda las imágenes subidas por el usuario.

Si no se sube ninguna imagen, asigna una imagen por defecto. Si se suben imágenes, las guarda en `public/uploads/` y registra la ruta en la base de datos.

MensajeController

`index()`

Muestra la bandeja de entrada del usuario con sus conversaciones recientes.

`contactar($idProducto)`

Abre un chat con el vendedor desde la página de detalle de un producto.

El usuario no escribe manualmente el ID del vendedor; el sistema lo obtiene desde el producto.

`ver($idContacto, $idProducto = null)`

Muestra una conversación concreta entre el usuario y otro contacto.

Si la conversación está asociada a un producto, también se tiene en cuenta ese producto.

`enviar()`

Guarda un nuevo mensaje en la base de datos.

Antes de enviarlo, comprueba que el usuario pueda escribir en esa conversación y que el mensaje no esté vacío.

`poll($idContacto = null, $idProducto = null)`

Devuelve las conversaciones y mensajes en formato JSON.

Se usa con JavaScript para actualizar la bandeja de mensajes cada pocos segundos sin recargar la página.

`cargarChat($idContacto, $idProducto, $productoTitulo, $contactoNombre)`

Carga los datos necesarios para mostrar un chat concreto.

Obtiene las conversaciones, los mensajes del hilo y los datos del contacto.

UsuarioController

`perfil()`

Muestra el perfil del usuario.

También permite actualizar datos básicos como el nombre y la dirección.

Además, muestra los productos publicados por el usuario.

ValoracionController

`index()`

Muestra las compras del usuario y permite ver cuáles ya han sido valoradas.

`crear($idCompra)`

Permite valorar al vendedor después de realizar una compra.

Comprueba que la compra sea válida, que pertenezca al usuario y que no haya sido valorada antes.

AdminController

`comprobarAdmin()`

Comprueba que el usuario tenga permisos de administrador.

Si no es administrador, lo redirige a la página principal.

`index()`

Muestra el panel básico de administración.

Desde ahí se pueden ver productos y usuarios.

`ocultarProducto($idProducto)`

Permite al administrador ocultar un producto cambiando su estado a eliminado.

`activarProducto($idProducto)`

Permite volver a poner un producto como disponible.

`bloquearUsuario($idUser)`

Bloquea un usuario cambiando su campo activo a 0.
También evita que el administrador se bloquee a sí mismo.
`activarUsuario($idUser)`
Vuelve a activar un usuario bloqueado cambiando su campo activo a 1.

Modelos

Modelo Usuario

`crear($data)`
Inserta un nuevo usuario en la base de datos.
La contraseña se guarda cifrada y el usuario se crea como activo por defecto.

`buscarPorEmail($email)`
Busca un usuario por su email.
Se utiliza principalmente en el inicio de sesión.

`buscarPorId($id)`
Busca un usuario por su ID.
Se utiliza para cargar datos del perfil.

`actualizarPerfil($id, $data)`
Actualiza los datos básicos del perfil del usuario.

`todos()`
Obtiene todos los usuarios registrados.
Se utiliza en el panel de administración.

`cambiarActivo($idUser, $activo)`
Activa o bloquea un usuario.

Modelo Producto

`todos($busqueda = "")`
Obtiene todos los productos disponibles.
Si hay búsqueda, filtra por título, descripción o categoría.

`porId($id)`
Busca un producto por su ID.
También obtiene datos del vendedor y su valoración media.

`porUsuario($idUser)`
Obtiene los productos publicados por un usuario.

crear(\$datos)

Crea un nuevo producto en la base de datos.

actualizar(\$idProducto, \$idUser, \$datos)

Actualiza los datos de un producto si pertenece al usuario actual.

eliminar(\$idProducto, \$idUser)

Marca un producto como eliminado.

marcarVendido(\$idProducto)

Cambia el estado del producto a vendido.

todosAdmin()

Obtiene todos los productos para el panel de administración.

cambiarEstadoAdmin(\$idProducto, \$estado)

Permite al administrador cambiar el estado de un producto.

validarProducto(\$datos)

Valida los datos de un producto antes de guardarlo.

Comprueba título, descripción, precio, categoría y estado.

Modelo Imagen

porProducto(\$idProducto)

Obtiene todas las imágenes asociadas a un producto.

Se usa en el detalle del producto y en el carrusel.

crear(\$idProducto, \$url)

Guarda en la base de datos la ruta de una imagen asociada a un producto.

Modelo Compra

crear(\$idComprador, \$idProducto, \$metodo)

Crea una compra en la base de datos.

También marca el producto como vendido. Usa una transacción para que ambas acciones se hagan correctamente.

porComprador(\$idComprador)

Obtiene todas las compras hechas por un usuario.

También incluye datos del producto, vendedor y valoración si existe.

buscarValorable(\$idCompra, \$idComprador)

Busca una compra concreta para comprobar si puede ser valorada.

Modelo Mensaje

conversaciones(\$idUser)



Obtiene las conversaciones recientes del usuario.
Muestra el contacto, producto relacionado, último mensaje y fecha.
hilo(\$usuarioA, \$usuarioB, \$idProducto = null)
Obtiene todos los mensajes entre dos usuarios.
Si hay producto asociado, filtra por ese producto.
enviar(\$emisor, \$receptor, \$contenido, \$idProducto = null)
Guarda un nuevo mensaje en la base de datos.
Comprueba que el mensaje no esté vacío y que el usuario no se mande mensajes a sí mismo.
existeConversacion(\$idUserio, \$idContacto, \$idProducto = null)
Comprueba si existe una conversación entre dos usuarios.

Modelo Valoracion

crear(\$idCompra, \$idEmisor, \$idReceptor, \$puntuacion, \$comentario = "")
Crea una valoración de un comprador hacia un vendedor.
Valida que la puntuación esté entre 1 y 5 y que el usuario no se valore a sí mismo.
existePorCompra(\$idCompra)
Comprueba si una compra ya tiene valoración.
porUsuario(\$idUserio)
Obtiene todas las valoraciones recibidas por un usuario.
ultimasPorUsuario(\$idUserio, \$limite = 3)
Obtiene las últimas valoraciones recibidas por un usuario.
Se usa en el detalle del producto para mostrar algunas opiniones del vendedor.
resumenUsuario(\$idUserio)
Calcula la valoración media y el número total de valoraciones de un usuario.

3.2.2.3. Cliente

La parte cliente de la aplicación está desarrollada con **HTML5, CSS3 y JavaScript**.

El cliente se encarga de:

- mostrar la interfaz al usuario;
- permitir la navegación entre secciones;
- mostrar formularios;

- validar algunos comportamientos básicos;
- actualizar partes de la página sin recargar;
- mostrar carruseles de imágenes;
- mejorar la experiencia de usuario.

La aplicación tiene un diseño adaptado a un marketplace de segunda mano, con una interfaz sencilla, clara y responsive.

3.2.2.3.1. Diseño de la interfaz

El diseño de la interfaz está basado en una estructura clara y sencilla, pensada para que el usuario pueda navegar fácilmente.

La aplicación incluye:

- cabecera con navegación;
- buscador de productos;
- listado de productos mediante tarjetas;
- página de detalle;
- formularios de login, registro, publicación y compra;
- perfil de usuario;
- sección de mensajes;
- panel básico de administración.

El color principal utilizado es el verde, asociado a la idea de sostenibilidad y mercado de segunda mano.

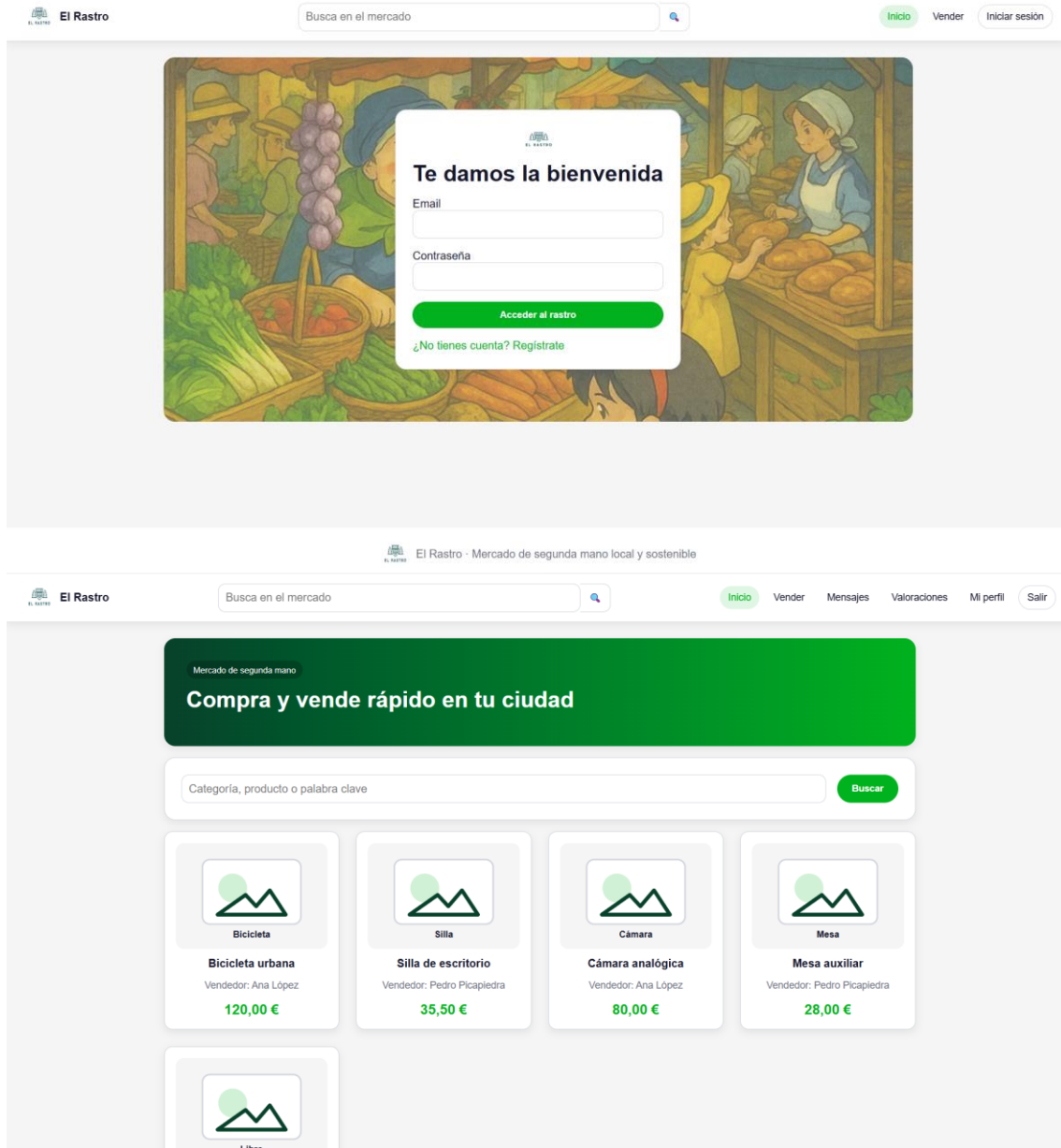
Las tarjetas de productos incluyen:

- imagen;
- título;
- precio;
- categoría;
- botón para ver detalle.

En la página de detalle se muestra:

- carrusel de imágenes;
- título;
- descripción;
- precio;
- estado del producto;
- vendedor;
- valoración del vendedor;
- botones de compra y contacto.

La interfaz utiliza bordes redondeados, sombras suaves y espaciado uniforme para mantener una apariencia limpia y moderna.




El Rastro



[Inicio](#)
[Vender](#)
[Mensajes](#)
[Valoraciones](#)
[Mi perfil](#)
[Salir](#)

Subir un producto

Título

Precio

Categoría

Selecciona una categoría

Estado del producto

Selecciona el estado

Descripción

Fotos del producto

Choose Files

No file chosen

Publicar


El Rastro



[Inicio](#)
[Vender](#)
[Mensajes](#)
[Valoraciones](#)
[Mi perfil](#)
[Salir](#)

El Rastro · Mercado de segunda mano local y sostenible

Ficha del producto



Bicicleta





Información del vendedor

Nombre	Ana López
Valoración	Sin valoraciones todavía
Ciudad / dirección	Cuenca
Categoría	Deportes
Estado del producto	Usado
Disponibilidad	disponible

[Ver opiniones](#)

[Comprar](#)

Bicicleta urbana

120,00 €

[Deportes](#)
[Usado](#)

Bicicleta de paseo en buen estado. Ideal para moverse por la ciudad.

51

Nicolas Rodríguez Jiménez


El Rastro

[Inicio](#)
[Vender](#)
[Mensajes](#)
[Valoraciones](#)
[Mi perfil](#)
[Salir](#)

Ficha del producto



Silla



Silla de escritorio

35,50 €

[Hogar](#) [Usado](#)

Silla cómoda con respaldo regulable. Presenta marcas de uso normales.

Información del vendedor

Nombre	Pedro Picapiedra
Valoración	5.0/5 · 1 opiniones
Ciudad / dirección	Madrid
Categoría	Hogar
Estado del producto	Usado
Disponibilidad	disponible

[Ocultar opiniones](#)

Últimas opiniones

★★★★★

"Todo perfecto, vendedor muy recomendable y comunicación rápida."

Por Ana López · Compra: Lámpara verde · 13/05/2026

[Contactar vendedor](#)

[Comprar](#)


El Rastro

[Inicio](#)
[Vender](#)
[Mensajes](#)
[Valoraciones](#)
[Mi perfil](#)
[Salir](#)

Confirmar compra

Vas a comprar **Silla de escritorio** por **35,50 €**.

Método de pago

Tarjeta

Titular de la tarjeta	Número de tarjeta
Ana López	0000 0000 0000 00
Caducidad	CVV
MM/AA	123

Por seguridad, la aplicación solo guarda el método de pago seleccionado, no los datos de la tarjeta.

[Confirmar compra](#)

[Cancelar](#)

 **El Rastro**

Busca en el mercado

[Inicio](#) [Vender](#) [Mensajes](#) [Valoraciones](#) [Mi perfil](#) [Salir](#)

Compra realizada correctamente. Ahora puedes valorar al vendedor.

Valorar vendedor

Compra: **Silla de escritorio**

Vendedor: **Pedro Picapiedra**

Puntuación

★★★★☆ 4 - Muy bien

Comentario opcional

Excelente trato, pero precio mejorable

[Guardar valoración](#)

[Ahora no](#)

 **El Rastro**

Busca en el mercado

[Inicio](#) [Vender](#) [Mensajes](#) [Valoraciones](#) [Mi perfil](#) [Salir](#)

Chats recientes

Pedro Picapiedra
Bicicleta urbana
Hola, ¿sigue disponible la bi...

Chat con Pedro Picapiedra

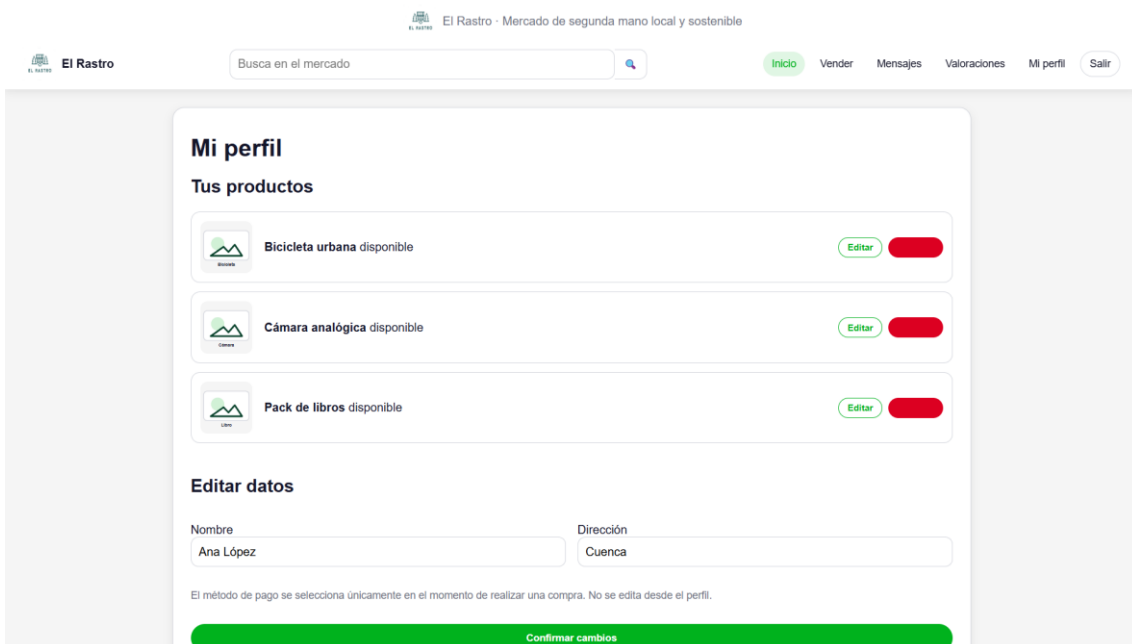
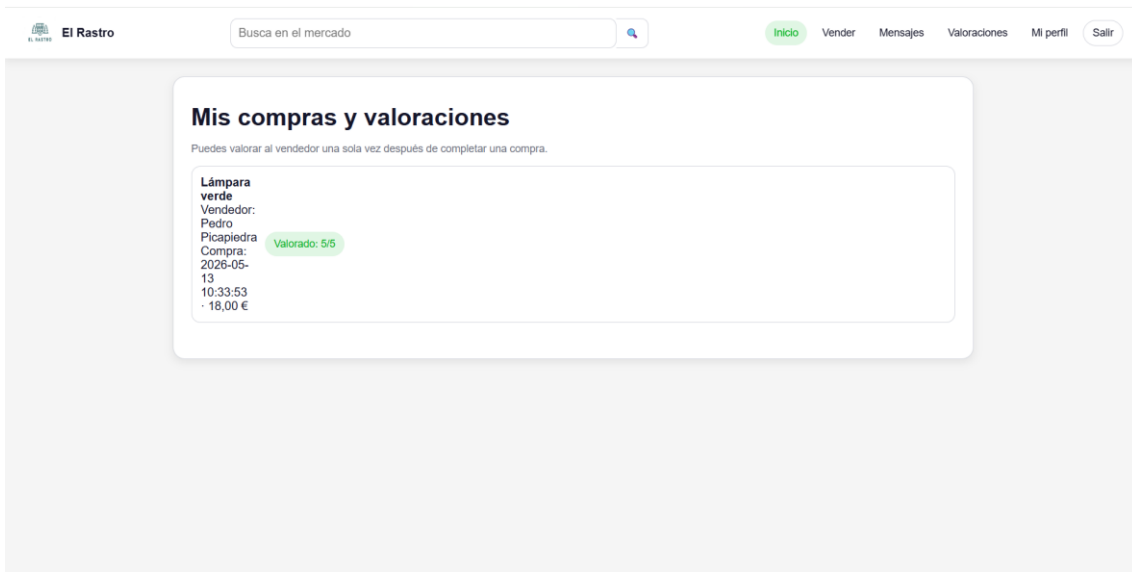
Producto: Bicicleta urbana

Pedro Picapiedra
Hola, ¿sigue disponible la bicicleta?
2026-05-13 10:33:53

Ana López
Sí, puedes verla esta tarde.
2026-05-13 10:33:53

Mensajes actualizados automáticamente.

Escribe un mensaje



3.2.2.3.2. Accesibilidad

- La aplicación tiene en cuenta varios aspectos básicos de accesibilidad.
- Los formularios utilizan etiquetas label asociadas a los campos, lo que facilita su comprensión.
- Los botones y enlaces tienen textos descriptivos
- Las imágenes incluyen atributo alt para describir su contenido.
- Los formularios utilizan campos obligatorios cuando es necesario

También se utilizan mensajes de error y éxito para informar al usuario de lo que ha ocurrido tras una acción.

Por ejemplo:

- credenciales incorrectas
- producto publicado correctamente
- compra realizada correctamente
- cuenta bloqueada
- mensaje enviado.

El diseño responsive permite que la aplicación pueda utilizarse en distintos tamaños de pantalla.

3.2.2.3.3. Usabilidad

La usabilidad de la aplicación se ha diseñado para que el usuario pueda realizar las acciones principales de forma sencilla.

Las acciones más importantes están visibles en el menú:

- inicio
- vender
- mensajes
- valoraciones
- perfil
- cerrar sesión

En la publicación de productos, las categorías y estados se seleccionan mediante desplegables, evitando que el usuario escriba valores incorrectos.

Además, el panel de administrador se mantiene muy básico para no complicar la aplicación, mostrando únicamente productos y usuarios con botones simples para activar, ocultar, bloquear o desbloquear.

3.2.2.3.4. Desarrollo web entorno cliente

La parte cliente utiliza JavaScript para añadir interactividad y mejorar la experiencia de usuario.

Las funcionalidades principales implementadas en cliente son:

- carrusel de imágenes en la ficha del producto;
- actualización automática de mensajes;
- cambios dinámicos en formularios de pago;

- despliegue de opiniones del vendedor;
- actualización del DOM sin recargar la página.

a. Formularios y su validación

La parte cliente utiliza JavaScript para añadir interactividad y mejorar la experiencia de usuario.

Las funcionalidades principales implementadas en cliente son:

- carrusel de imágenes en la ficha del producto;
- actualización automática de mensajes;
- cambios dinámicos en formularios de pago;
- despliegue de opiniones del vendedor;
- actualización del DOM sin recargar la página.

b. Manejo y gestión de eventos: Teclado, ratón y estados de la ventana

JavaScript se utiliza para gestionar eventos del usuario.

Entre los eventos principales se encuentran:

- clic en botones del carrusel;
- clic en miniaturas de imágenes;
- clic en “Ver opiniones”;
- cambio de método de pago;
- envío de mensajes;
- actualización automática mediante intervalos.

En el carrusel se gestionan eventos de clic para avanzar o retroceder entre imágenes.

En el formulario de compra se detecta el cambio del método de pago seleccionado para mostrar los campos correspondientes.

En la mensajería se utiliza un temporizador con:

`setInterval()`

para actualizar los mensajes cada pocos segundos.

c. Gestión y almacenamiento de datos e información en el cliente

La aplicación no almacena datos sensibles en el cliente.

Los datos importantes, como usuarios, productos, compras, mensajes y valoraciones, se almacenan en la base de datos del servidor.

En el cliente solo se utilizan datos temporales para mejorar la interacción, por ejemplo:

- imagen actual del carrusel;
- chat abierto;
- método de pago seleccionado;
- datos recibidos por AJAX para actualizar mensajes.

La sesión de usuario se gestiona en el servidor mediante PHP y `$_SESSION`

d. Modificación del DOM

JavaScript modifica el DOM para actualizar partes de la interfaz sin recargar toda la página.

Algunos ejemplos son:

- cambiar la imagen principal del carrusel;
- marcar una miniatura como activa;
- mostrar u ocultar opiniones del vendedor;
- mostrar campos diferentes según el método de pago;
- actualizar el listado de conversaciones;
- actualizar los mensajes del chat abierto.

En la mensajería, el servidor devuelve datos en JSON y JavaScript actualiza la zona de chats y mensajes.

Esto mejora la experiencia de usuario porque evita recargar manualmente la página.

e. Animaciones, efectos, cambios dinámicos de estilos etc... para dinamizar la parte visible al cliente

La aplicación utiliza efectos visuales sencillos para mejorar la interfaz sin sobrecargarla. Entre ellos:

- efectos hover en botones;
- sombras suaves en tarjetas;
- bordes redondeados;
- estados activos en botones o miniaturas;
- aparición de secciones dinámicas como opiniones o campos de pago.

El carrusel de imágenes permite cambiar visualmente entre imágenes del producto.

Los botones cambian de aspecto al pasar el ratón, ayudando al usuario a identificar elementos interactivos.

Se ha optado por una interfaz simple para mantener el proyecto claro y fácil de mantener.

f. Comunicación AJAX

La comunicación AJAX se utiliza principalmente en la mensajería.

El cliente realiza peticiones con `fetch()` a una ruta del servidor:

`mensaje/poll`

o, si hay un chat abierto:

`mensaje/poll/idContacto/idProducto`

El servidor responde con datos en formato JSON:

```
{  
  "ok": true,  
  "usuario_id": 1,  
  "conversaciones": [],  
  "mensajes": []  
}
```

JavaScript recibe esa información y actualiza la bandeja de entrada y el hilo del chat.

Esta comunicación permite que la pantalla de mensajes se actualice automáticamente sin que el usuario tenga que recargar la página.

g. Comunicación asíncrona con el servidor

La comunicación asíncrona se implementa mediante polling.

El sistema hace una petición cada pocos segundos al servidor para comprobar si hay mensajes nuevos.

El funcionamiento es:

JavaScript espera unos segundos

→ hace `fetch` al servidor

→ el servidor consulta la base de datos

→ devuelve JSON

→ JavaScript actualiza el HTML

Esto da una sensación de mensajería directa.

Aunque no es tiempo real real como WebSockets, es una solución adecuada para un proyecto DAW porque es más sencilla de implementar y suficiente para simular una bandeja de entrada dinámica.

La ventaja de este sistema es que mejora la experiencia del usuario manteniendo una implementación comprensible y adecuada al nivel del proyecto.

3.2.3. Fase de despliegue

La fase de despliegue consiste en preparar la aplicación para que pueda ejecutarse correctamente en un entorno distinto al de desarrollo del código.

En este proyecto, el despliegue se realizará en un **equipo local propio utilizando XAMPP**, ya que la aplicación necesita un servidor web, PHP y una base de datos MySQL/MariaDB.

XAMPP facilita este proceso porque incluye en una sola instalación las herramientas necesarias:

- Apache
- PHP
- MySQL/MariaDB
- phpMyAdmin

La aplicación **El Rastro** está desarrollada en PHP y utiliza una base de datos MySQL, por lo que no sería suficiente con tener únicamente Apache instalado. También es necesario tener activo el servicio de base de datos.

3.2.3.1. Despliegue utilizando un hosting

Este apartado no se desarrolla en profundidad porque el despliegue final del proyecto se realizará en un entorno local propio utilizando XAMPP.

En caso de desplegar la aplicación en un hosting real, sería necesario contratar un servicio compatible con PHP y MySQL, crear una base de datos en el panel del hosting, importar el archivo SQL del proyecto, subir los archivos de la aplicación y modificar los datos de conexión a la base de datos.

Sin embargo, para este proyecto se ha optado por el despliegue local, ya que permite probar todas las funcionalidades de la aplicación sin necesidad de contratar servicios externos.

3.2.3.2. Despliegue en un equipo local propio

El despliegue en local se realizará utilizando **XAMPP**, que permite ejecutar la aplicación en un ordenador personal.

El proceso de despliegue será el siguiente:

1. Instalar XAMPP en el equipo.
2. Copiar la carpeta del proyecto dentro de htdocs.
3. Iniciar Apache y MySQL desde el panel de XAMPP.
4. Crear la base de datos desde phpMyAdmin.
5. Importar el archivo SQL del proyecto.
6. Revisar los datos de conexión de la aplicación.
7. Acceder a la aplicación desde el navegador.
8. Probar las funcionalidades principales.

La carpeta del proyecto debe colocarse en la siguiente ruta:

C:\xampp\htdocs\el_rastro_mvc

La aplicación se ejecutará desde el navegador mediante la URL:

http://localhost/el_rastro_mvc/public/index.php

La base de datos se creará desde phpMyAdmin, accediendo a:

<http://localhost/phpmyadmin>

Desde ahí se importará el archivo SQL del proyecto:

database/el_rastro.sql

Este archivo contiene la creación de las tablas necesarias y los datos iniciales de prueba.

La configuración de conexión a la base de datos debe coincidir con los datos del entorno local.

Una vez importada la base de datos y copiado el proyecto en htdocs, se iniciarán los servicios de **Apache** y **MySQL** desde el panel de XAMPP.

3.3. Seguimiento y control de incidencias

Durante el desarrollo y despliegue del proyecto se ha realizado un seguimiento de las incidencias detectadas para corregir errores y mejorar el funcionamiento de la aplicación.

Las incidencias encontradas se han ido resolviendo de forma progresiva, comprobando cada funcionalidad después de aplicar los cambios.

Algunas incidencias habituales durante el desarrollo han sido:

- errores de conexión con la base de datos
- rutas incorrectas en la aplicación
- problemas al cargar imágenes
- errores en formularios
- fallos en consultas SQL
- problemas de sesión de usuario
- errores al enviar mensajes
- problemas al comprar productos
- errores de permisos entre usuarios

Para controlar estas incidencias se ha seguido un proceso sencillo:

1. Detectar el error durante las pruebas.
2. Identificar en qué parte de la aplicación ocurre.
3. Revisar el archivo relacionado.
4. Corregir el código o la consulta SQL.
5. Probar de nuevo la funcionalidad.
6. Comprobar que el cambio no afecta a otras partes.

También se han utilizado mensajes de error y confirmación dentro de la aplicación para informar al usuario del resultado de cada acción.

Algunos ejemplos son:

- Credenciales incorrectas.
- Producto publicado correctamente.
- Mensaje enviado.
- Compra realizada correctamente.
- No tienes permiso para acceder a esta sección.

El control de incidencias ha permitido mejorar la estabilidad del proyecto y asegurar que las funcionalidades principales funcionen correctamente antes de la entrega.

3.4. Indicadores de calidad de procesos

Para comprobar la calidad del desarrollo se han tenido en cuenta distintos indicadores relacionados con el funcionamiento, la organización del código, la seguridad básica y la experiencia de usuario.

Los principales indicadores de calidad del proyecto son:

- La aplicación se ejecuta correctamente en XAMPP.
- La conexión con la base de datos funciona.
- Las tablas se crean correctamente al importar el SQL.
- Los formularios validan los datos introducidos.
- Las contraseñas se almacenan cifradas.
- Las consultas a la base de datos utilizan PDO.
- Los usuarios no pueden acceder a zonas privadas sin iniciar sesión.
- Los usuarios no pueden editar productos de otros usuarios.
- Los productos vendidos no se eliminan físicamente de la base de datos.
- La mensajería se actualiza automáticamente cada pocos segundos.
- El carrusel de imágenes funciona correctamente.

La interfaz es clara y se adapta a distintos tamaños de pantalla.

También se han probado los principales flujos de uso de la aplicación:

- registro de usuario
- inicio de sesión
- cierre de sesión
- publicación de producto
- edición de producto
- eliminación lógica de producto
- búsqueda de productos
- visualización del detalle del producto
- contacto con el vendedor
- envío y recepción de mensajes
- compra de producto
- valoración del vendedor
- bloqueo y desbloqueo de usuarios desde administración
- ocultación y activación de productos desde administración

Estos indicadores permiten comprobar que la aplicación cumple los requisitos establecidos y que las funcionalidades principales se ejecutan correctamente.

Además, la organización mediante MVC permite que el código esté separado en modelos, vistas y controladores, lo que facilita su mantenimiento y futuras ampliaciones.

4. Recursos materiales

Para el desarrollo del proyecto se han utilizado distintos recursos materiales relacionados principalmente con el desarrollo software, el diseño de la interfaz, la gestión de la base de datos y las pruebas en entorno local.

Al tratarse de una aplicación web desarrollada como proyecto académico, la mayoría de herramientas utilizadas son gratuitas o de uso libre, por lo que el coste económico real del proyecto es reducido.

4.1. Inventario, valorado de medios

Recurso	Descripción	Coste aproximado
Ordenador personal	Equipo utilizado para desarrollar, probar y documentar la aplicación	700 €
Conexión a Internet	Necesaria para consultar documentación, descargar herramientas y probar recursos	30 €/mes
XAMPP	Entorno local con Apache, PHP y MySQL	0 €
Visual Studio Code	Editor de código utilizado para programar la aplicación	0 €
MySQL	Sistema gestor de base de datos	0 €
PHP	Lenguaje de programación del servidor	0 €
HTML5, CSS3 y JavaScript	Tecnologías utilizadas en la parte cliente	0 €
Figma	Herramienta utilizada para prototipado y diseño visual	0 €
GitHub	Repositorio para guardar y controlar versiones del proyecto	0 €

4.2. Presupuesto económico

El presupuesto económico del proyecto se ha calculado teniendo en cuenta los recursos utilizados durante el desarrollo. Al tratarse de un proyecto académico, no se ha contratado personal externo ni servicios de pago.

Aunque el valor total estimado es de 730 €, la inversión real ha sido menor, ya que el ordenador y la conexión a Internet ya estaban disponibles antes del inicio del proyecto. Si la aplicación se desplegara en un entorno real, habría que añadir costes de hosting, dominio, mantenimiento y posibles copias de seguridad.

5. Recursos humanos

El proyecto ha sido desarrollado por una única persona, asumiendo todas las funciones necesarias para su realización.

Durante el desarrollo se han realizado tareas propias de distintos perfiles profesionales, aunque todas han sido ejecutadas por el mismo alumno.

5.1. Organización

La organización del trabajo se ha dividido en varias fases.

Primero se realizó el análisis inicial del proyecto, definiendo la idea principal, los usuarios, los requisitos y las funcionalidades básicas.

A continuación se decidió el diseño visual y guía de estilos

Después se diseñó la base de datos, estableciendo las entidades principales de la aplicación: usuarios, productos, imágenes, compras, mensajes y valoraciones.

Posteriormente se desarrolló la aplicación siguiendo una estructura MVC, separando modelos, vistas y controladores.

5.2. Contratación

No ha sido necesario realizar ninguna contratación externa.
Todas las tareas han sido desarrolladas por el propio alumno, utilizando herramientas gratuitas y recursos disponibles.

5.3. Prevención de riesgos laborales

Aunque se trata de un proyecto de desarrollo software y no implica riesgos físicos importantes, sí existen algunos riesgos asociados al trabajo prolongado con equipos informáticos.

Los principales riesgos son:

- fatiga visual
- malas posturas
- dolor de espalda o cuello
- cansancio mental
- uso prolongado del teclado y ratón

6. Viabilidad técnica

El proyecto es técnicamente viable porque utiliza tecnologías conocidas, estables y ampliamente utilizadas en el desarrollo web.

Las tecnologías principales son:

- PHP
- MySQL
- HTML5
- CSS3

- JavaScript
- Apache
- PDO

Todas estas tecnologías pueden ejecutarse en un entorno local mediante XAMPP, WAMP o Laragon, por lo que no es necesario contratar un servidor externo para probar la aplicación.

Además, la arquitectura MVC facilita el mantenimiento del código, ya que separa la lógica de negocio, el acceso a datos y la interfaz.

6.1. Estudio de viabilidad técnica

Desde el punto de vista técnico, el proyecto se puede desarrollar y ejecutar correctamente con los recursos disponibles.

PHP permite gestionar la lógica del servidor, las sesiones y la conexión con la base de datos. MySQL permite almacenar la información de usuarios, productos, imágenes, compras, mensajes y valoraciones. JavaScript permite añadir interactividad, como el carrusel de imágenes y la actualización automática de mensajes.

El uso de PDO permite trabajar con consultas preparadas, lo que mejora la seguridad frente a inyección SQL.

Por tanto, el proyecto es viable técnicamente y puede ejecutarse en un servidor local sin necesidad de grandes recursos.

7. Viabilidad económica-financiera

La viabilidad económica del proyecto es positiva, ya que se ha desarrollado utilizando herramientas gratuitas y recursos ya disponibles.

No se han requerido licencias de pago ni contratación de servicios externos.

El coste principal corresponde al equipo informático y a la conexión a Internet, pero ambos recursos ya estaban disponibles antes del desarrollo.

En caso de querer publicar la aplicación en Internet, habría que considerar costes adicionales como:

- dominio
- hosting
- certificado SSL

- copias de seguridad
- mantenimiento

Aun así, estos costes serían moderados para una aplicación pequeña.

7.1. Inversiones y gastos

Como ya se ha mencionado anteriormente, los únicos gastos/inversiones reales del proyecto hasta ahora son el equipo personal (pc, unos 700 euros) y la conexión a internet.

7.2. Financiación

El proyecto no ha necesitado financiación externa para su desarrollo inicial, ya que se ha realizado con recursos propios del alumno y utilizando herramientas gratuitas.

En una primera fase, la aplicación funcionaría sin coste para los usuarios, con el objetivo de conseguir una base inicial de personas registradas y productos publicados.

En caso de llevar la aplicación a un entorno real, la financiación podría obtenerse mediante dos vías principales:

- publicidad dentro de la plataforma
- paquetes de pago para destacar anuncios

La publicidad permitiría obtener ingresos mostrando anuncios en zonas concretas de la web, sin afectar demasiado a la experiencia del usuario.

Por otro lado, cuando la plataforma alcanzase un número mínimo de usuarios habituales, se podrían ofrecer paquetes de pago para destacar productos. Estos paquetes permitirían que un anuncio apareciera en posiciones más visibles del listado o durante más tiempo en la página principal.

De esta forma, la aplicación podría empezar siendo gratuita y más adelante incorporar opciones de monetización sin impedir el uso básico de la plataforma.

7.3. Viabilidad económico-financiera

El proyecto es económicamente viable porque el coste de desarrollo inicial es bajo y no requiere una gran inversión.

La aplicación puede funcionar en local sin coste adicional y, en caso de publicarse en Internet, podría desplegarse en un hosting básico con un coste reducido.

Además, el modelo de ingresos podría basarse en dos líneas principales:

- publicidad
- anuncios destacados de pago

La publicidad serviría como fuente de ingresos inicial, especialmente si la plataforma empieza a recibir visitas de forma constante.

Más adelante, cuando exista una cantidad suficiente de usuarios activos, se podrían ofrecer paquetes para destacar anuncios. Esta opción permitiría a los vendedores aumentar la visibilidad de sus productos pagando una pequeña cantidad.

Este sistema permitiría mantener gratuita la publicación normal de productos y obtener ingresos únicamente de los usuarios que quieran mejorar la visibilidad de sus anuncios.

En conclusión, la viabilidad económico-financiera es positiva, ya que el proyecto tiene costes reducidos y cuenta con posibles vías de monetización escalables en caso de crecimiento de la plataforma.

8. Conclusión

El proyecto **El Rastro** ha consistido en el desarrollo de una aplicación web tipo marketplace orientada a la compraventa de productos de segunda mano entre usuarios. La finalidad principal ha sido crear una plataforma sencilla, funcional y accesible, donde cualquier usuario pueda registrarse, publicar productos, contactar con vendedores, realizar compras y valorar la experiencia después de una transacción.

Durante el desarrollo se han aplicado los conocimientos adquiridos en el ciclo de **Desarrollo de Aplicaciones Web**, utilizando tecnologías como **PHP, MySQL, HTML5, CSS3 y JavaScript**. La aplicación se ha estructurado siguiendo el patrón **MVC**, separando la lógica de negocio, el acceso a datos y la interfaz de usuario. Esto ha permitido organizar mejor el código y facilitar futuras mejoras del proyecto.

En la parte del servidor se han implementado funcionalidades como el registro e inicio de sesión, la gestión de productos, la subida de imágenes, las compras, las valoraciones, la mensajería entre usuarios y un panel básico de administración. Además, se ha utilizado **PDO** para la conexión con la base de datos, trabajando con consultas preparadas para mejorar la seguridad.

En la parte cliente se ha desarrollado una interfaz clara y responsive, adaptada a distintos tamaños de pantalla. También se han añadido funcionalidades dinámicas mediante JavaScript, como el carrusel de imágenes en el detalle del producto, el cambio de campos según el método de pago y la actualización automática de los mensajes para simular una experiencia de chat más directa.

A nivel de base de datos, se ha diseñado un modelo relacional que permite almacenar usuarios, productos, imágenes, compras, mensajes y valoraciones. Esto permite mantener el historial de operaciones de la plataforma y evita perder información importante cuando un producto se vende o se elimina de forma lógica.

El proyecto cumple con los objetivos planteados inicialmente, ya que permite realizar las acciones principales de una plataforma de compraventa: publicar productos, buscar anuncios, consultar detalles, contactar con vendedores, comprar productos y valorar usuarios. Además, se han añadido mejoras como el panel de administración y el sistema de opiniones, que aportan más control y confianza a la aplicación.

Como posibles mejoras futuras, se podrían incorporar funcionalidades como notificaciones reales en tiempo real, filtros de búsqueda más avanzados, geolocalización de productos, pasarela de pago real, sistema de favoritos y opciones de monetización mediante publicidad o anuncios destacados.

En conclusión, **El Rastro** es una aplicación web funcional y ampliable, que cumple los requisitos principales del proyecto y demuestra la integración de diferentes tecnologías tanto del lado servidor como del lado cliente.

Bibliografía/Webgrafía

Documentación de PHP

PHP Documentation. *Manual oficial de PHP*.

Disponible en: [página oficial de PHP](https://www.php.net/manual/).

Consultado para el uso de funciones del lenguaje, gestión de sesiones,

formularios, conexión con base de datos mediante PDO y funciones de seguridad como `password_hash()` y `password_verify()`. PHP recomienda `password_hash()` para crear hashes seguros de contraseñas mediante algoritmos fuertes de un solo sentido, y también indica que una columna de 255 caracteres es una buena elección para almacenarlos.

Documentación de PDO

PHP Documentation. *PDO::prepare*.

Disponible en: manual oficial de PHP.

Consultado para el uso de consultas preparadas con PDO en el acceso a la base de datos.

La documentación oficial explica que `PDO::prepare()` prepara una sentencia SQL para ejecutarla posteriormente con `execute()` y que los marcadores de parámetros sirven para sustituir valores de forma segura, evitando incluir directamente datos del usuario dentro de la consulta.

Documentación de MySQL

MySQL. *MySQL 8.0 Reference Manual*.

Disponible en: documentación oficial de MySQL.

Consultado para el diseño de tablas, claves primarias, claves foráneas, restricciones de integridad y relaciones entre tablas.

La documentación de MySQL explica que las claves foráneas permiten relacionar tablas y mantener la consistencia de los datos mediante restricciones referenciales.

Documentación de HTML, CSS y JavaScript

MDN Web Docs. *HTML, CSS y JavaScript*.

Disponible en: documentación de MDN Web Docs.

Consultado para la estructura de páginas HTML, estilos CSS, eventos de JavaScript, manipulación del DOM y uso de formularios.

Documentación de Fetch API

MDN Web Docs. *Fetch API*.

Disponible en: documentación de MDN Web Docs.

Consultado para la actualización automática de mensajes mediante peticiones asíncronas desde JavaScript.
MDN explica que la Fetch API permite realizar peticiones para obtener recursos a través de la red y que el método `fetch()` devuelve una promesa con la respuesta del servidor.

XAMPP

Apache Friends. *XAMPP*.

Disponible en: página oficial de Apache Friends.

Consultado para la instalación y uso del entorno local de desarrollo con Apache, PHP, MySQL/MariaDB y phpMyAdmin.

GitHub

GitHub. *Documentación de GitHub*.

Disponible en: documentación oficial de GitHub.

Consultado para la creación del repositorio, subida del proyecto y control básico de versiones.

Figma

Figma. *Herramienta de diseño y prototipado*.

Utilizado para la creación del prototipo visual inicial de la aplicación.

Recursos propios

Además de la documentación técnica, se han utilizado apuntes del ciclo de Desarrollo de Aplicaciones Web y recursos propios elaborados durante el desarrollo del proyecto.

Anexos

Anexo I. Posibles mejoras futuras

Como futuras mejoras del proyecto se podrían incluir:

- Sistema de favoritos
- Filtros avanzados por precio, categoría o ubicación
- Notificaciones en tiempo real
- Pasarela de pago real
- Geolocalización de productos
- Recuperación de contraseña
- Sistema de reportes
- Panel de administración más completo
- Publicidad dentro de la plataforma
- Paquetes de pago para destacar anuncios

Anexo II. Tecnologías utilizadas

- PHP
- MySQL
- PDO
- HTML5
- CSS3
- JavaScript
- XAMPP
- Apache
- phpMyAdmin
- GitHub
- Figma

PHP se utiliza para la parte servidor.
MySQL se utiliza como sistema gestor de base de datos.
HTML, CSS y JavaScript se utilizan para la parte cliente.
XAMPP se utiliza como entorno local de despliegue.
GitHub se utiliza como repositorio del proyecto.

Anexo III. Funcionalidades principales



La aplicación permite realizar las siguientes acciones:

- Registro de usuarios
- Inicio y cierre de sesión
- Publicación de productos
- Subida de imágenes
- Listado y búsqueda de productos
- Detalle de producto con carrusel
- Compra simulada de productos
- Mensajería entre usuarios
- Actualización automática de mensajes
- Valoración de vendedores
- Gestión de perfil
- Panel básico de administración
- Bloqueo y desbloqueo de usuarios
- Ocultación y activación de productos